The Journal of Systems and Software 231 (2026) 112636

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

BugMentor: Generating answers to follow-up questions from software bug reports using structured information retrieval and neural text generation ☆

Usmi Mukherjee *, Mohammad Masudur Rahman

Dalhousie University, Halifax, Nova Scotia, Canada

ARTICLE INFO

Dataset link: http://bit.ly/3I8l86a

Keywords:
Software engineering
Software bugs
Follow-up questions
Neural text generation
Information retrieval

ABSTRACT

Software bug reports often lack crucial information (e.g., steps to reproduce), which makes bug resolution challenging. Developers thus ask follow-up questions to capture additional information. However, according to existing evidence, bug reporters often face difficulties answering them, which leads to the premature closing of bug reports without any resolution. Recent studies suggest follow-up questions to support the developers, but answering the follow-up questions still remains a major challenge. In this paper, we propose BugMentor, a novel approach that combines structured information retrieval and neural text generation (e.g., Mistral) to generate appropriate answers to the follow-up questions. Our technique identifies the past relevant bug reports to a given bug report, captures contextual information, and then leverages it to generate the answers. We evaluate our generated answers against the ground truth answers using four appropriate metrics, including BLEU Score and Semantic Similarity. We achieve a BLEU Score of up to 72 and Semantic Similarity of up to 92 indicating that our technique can generate understandable and good answers to the follow-up questions according to Google's AutoML Translation documentation. Our technique also outperforms four existing baselines with a statistically significant margin. We also conduct a developer study involving 23 participants where the answers from our technique were found to be more accurate, more precise, more concise and more useful.

1. Introduction

Software bugs are human-made errors in a software system that prevent the software from working as expected (IEEE, 1990). Studies have shown that software bugs cost the global economy billions of dollars every year (Britton et al., 2013; Zou et al., 2018). Developers also spend ~50% of their programming time finding and fixing bugs (Britton et al., 2013). Thus, bug resolution has been one of the major challenges in software maintenance (Zou et al., 2018). Hundreds of software bugs are submitted to bug-tracking systems like GitHub and JIRA as *bug reports* (Anvik et al., 2006). These bugs are then triaged, analyzed, and resolved by developers.

Ideally, bug reports should contain all the information, such as system configuration, expected behavior, observed behavior, and reproducing steps that help a developer resolve a bug (Chaparro et al., 2017). However, in practice, they often do not contain all the required information for reproducing or resolving the bug (Chaparro et al., 2017). According to existing literature (Chaparro et al., 2017), 64.8% of bug reports do not contain any expected behavior of target software systems, and 48.6% of them do not explicitly describe the steps to reproduce a bug. Missing information like this has been found to be a key factor behind the non-reproducibility of software bugs (Rahman et al., 2020). In a survey conducted by Zou et al. (2018), 77% of 327 professional developers from the major technology companies (e.g., Google, Meta, Amazon, Microsoft) consider missing information as a major problem and emphasize on complementing bug reports with useful information (e.g., steps to reproduce, environmental configuration) (Zou et al., 2018). In short, missing information has been a key challenge toward cost-effective bug resolution.

Many software projects on GitHub now require bug reports to adhere to specific templates or standard guidelines[1]. Unfortunately, many bug reporters might fail to comply with them and might not be able to provide all the information during report submission Imran et al. (2021). Developers thus often pose *follow-up* questions to bug reporters soliciting the missing information. Unfortunately, the bug reporters often find it challenging to answer the follow-up questions in a timely fashion, according to a recent developer survey (Rahman et al., 2020). For instance, Fig. 1 shows how a bug report was closed prematurely without any resolution due to a lack of responses to the follow-up question posed by the developer.

☆ Editor: Dr Leandro Minku.

* Corresponding author.
E-mail addresses: usmi.mukherjee@dal.ca (U. Mukherjee), mh264771@dal.ca (M.M. Rahman).

[1] https://tinyurl.com/5n8sehce

https://doi.org/10.1016/j.jss.2025.112636
Received 21 October 2024; Received in revised form 4 September 2025; Accepted 12 September 2025
Available online 29 September 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

Over the last few decades, extensive research has been conducted to support various bug report management tasks, including bug triage (Zhao et al., 2019; Bodden et al., 2017), duplicate bug detection (Nguyen et al., 2012; Chaparro et al., 2019), bug localization (Zhang et al., 2019; Xiao et al., 2019), and bug reproduction (Johnson et al., 2022). These studies have significantly advanced our understanding of the challenges in bug report management. However, important communications within bug reports, especially follow-up questions and answers between bug reporters and developers, have not been exhaustively investigated to date. Breu et al. (2010) were the first to analyze follow-up questions from bug reports quantitatively and qualitatively. Their study revealed that 32.34% of these questions were never responded to, which is critical for effective bug triaging, finding, and reproduction. More recently, Imran et al. (2021) automatically recommend follow-up questions for deficient bug reports employing information retrieval. While these studies have made valuable contributions by analyzing or recommending the follow-up questions, they do not answer them, which remains a major challenge for bug reporters (Rahman et al., 2020). In this work, we attempt to bridge this gap by automatically answering the follow-up questions where the bug reporters struggle. Our goal is to add complementary information to deficient bug reports automatically and thus support the bug report management tasks.

Automated Question Answering (QA) has been an active research topic for decades in Information Retrieval (IR) and Natural Language Processing (NLP) communities (Ravichandran and Hovy, 2002; Brill et al., 2002; Waltz, 1978; Iyyer et al., 2014; Asaduzzaman et al., 2013; Tian et al., 2017; Lu et al., 2021; Bansal et al., 2021; Xu et al., 2017; Abdellatif et al., 2020). There also have been several works in the context of software engineering. Tian et al. (2017) designed APIBot that can answer questions related to an API by analyzing relevant API documentation. Bansal et al. (2021) designed a context-aware QA system to answer basic questions about subroutines. Lu et al. (2021) proposed a QA approach that can provide answers by executing structured queries generated from bug templates. However, their approach might fail when a bug report does not contain the requested information. Xu et al. (2017) designed AnswerBot that can synthesize answers for a non-factoid technical question from StackOverflow Q&A website. While the above approaches are a source of inspiration, they do not answer the follow-up questions posed by developers against the bug reports.

In this paper, we propose a novel technique — *BugMentor* — that can offer relevant answers to follow-up questions from bug reports by combining structured information retrieval and neural text generation. First, we capture textually relevant questions, answers, and bug reports against a follow-up question using structured information retrieval (Saha et al., 2013). Then we capture each item's embeddings using CodeBERT (Feng et al., 2020) and re-rank them based on their semantic relevance to the question. Second, we leverage the ranked items above as *context* with a neural text-generation technique (e.g., Mistral, Llama) and generate meaningful answers to the follow-up question (Yu et al., 2022).

We select the top 20 popular projects from GitHub that use four programming languages (e.g. Java, Python, JavaScript and C++) and collect 30,869 bug reports from them for our experiments. We evaluate our technique using four popular metrics for text generation, namely BLEU score (Papineni et al., 2002), METEOR (Banerjee and Lavie, 2005), Semantic Similarity (Haque et al., 2022), and ROUGE (Lin, 2004). We achieve a BLEU score of 36.18 which indicates that our generated answers are *understandable to good* according to Google AutoML documentation [2]. We also conduct an ablation study to justify our combination of structured information retrieval and neural text generation in BugMentor. We find that BugMentor leverages the rich context captured through structured information retrieval and thus can generate meaningful answers. BugMentor also outperforms all four baselines — BM25 (McCandless et al., 2010), Mistral (Jiang et al., 2023), AnswerBot (Xu et al., 2017), DialBERT (Li et al., 2020) in all four metrics. To further demonstrate its benefit, we conduct a developer study involving 23 participants. According to the participants, the answers from BugMentor were more accurate, precise, concise and useful compared to the baseline answers.

To summarize, we make three contributions in this work:

(a) A novel technique — BugMentor — that can generate relevant answers to follow-up questions from bug reports by combining structured information retrieval and neural text generation (e.g., Mistral).
(b) A comprehensive evaluation and validation of BugMentor using both popular performance metrics (e.g., BLEU score, METEOR score, ROUGE, Semantic Similarity) and a developer study involving 23 participants.
(c) A replication package that includes our working prototype, experimental dataset, and other configuration details for the replication or third-party reuse by the community (Mukherjee and Rahman, 2025).

2. Motivating example

To demonstrate the potential benefit of our work, let us consider the example bug report in Fig. 1. It has been taken from the *tensorflow* repository on GitHub[3]. The bug report (Step (a), Fig. 1) discusses a memory allocation issue and provides minimal steps to reproduce the issue. However, it does not follow the standard issue template suggested by GitHub https://github.com/tensorflow/tensorflow/blob/master/ISSUE_TEMPLATE.md and lacks sufficient information about the system configuration. Subsequently, the developer (*@mohantym*) requests the missing information as a follow-up question that focuses on the occurrence of the issue on the *nightly* version of the software (Step (b), Fig. 1). However, neither the bug reporter (*@DanielZanchi*) nor anyone else provided an answer to the follow-up question. As a result, the bug report was initially marked as *stale* (Step (c), Fig. 1) and later closed due to a lack of response (Step (d), Fig. 1).

As shown in Fig. 1, without proper support in gathering missing information, software bugs either take longer to be resolved or remain unresolved and are ultimately closed. Several bugs, such as memory allocation bugs, are considered to be severe (Li et al., 2006) that significantly impact software quality. Our work — BugMentor — delivers meaningful answers to such follow-up questions.

To illustrate the effectiveness of BugMentor, let us examine the bug report Fig. 2 submitted to the *TypeScript* repository on GitHub[4]. The bug report discusses an issue where TypeScript compiler fails to throw an error when encountering generic type parameters in JavaScript code. TypeScript compiler can compile both JavaScript and TypeScript code. However, generic type parameters are TypeScript features and are not allowed in JavaScript code. The expected behavior for TypeScript is to issue an error in this context, as generic type parameters should not be allowed in JavaScript code. However, the actual behavior is that no error is thrown (Step (a), Fig. 2). The developer asks a follow-up question inquiring about TypeScript's handling of the type parameter in this context, whether it simply ignores the type parameter or if it contributes to type inference and checking in some way(Step (b), Fig. 2).

BugMentor's response provides valuable insights into the issue at hand in Table 1. It acknowledges that while TypeScript can be used to validate JavaScript code, it currently fails to issue errors when encountering invalid features such as type parameters in JavaScript

[2] https://cloud.google.com/translate/automl/docs/evaluate

[3] https://github.com/tensorflow/tensorflow/issues/58280

[4] https://github.com/microsoft/TypeScript/issues/31683

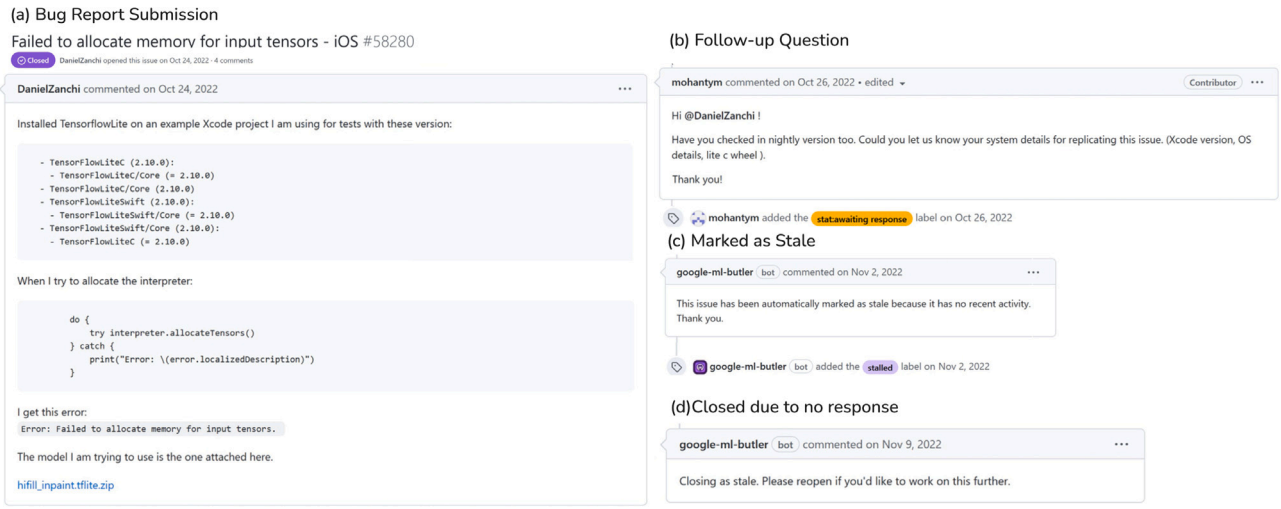

Fig. 1. An example of a bug report (ID #58280) being closed due to a lack of response to the follow-up question.

Table 1
Bug report explanation.

Bug Report #31683
(a) Answer: TypeScript should validate constructs in JS files but throw an error for TS-specific features like type parameters. In this case, the type parameter is not ignored; instead, a new type variable is created within the static method's scope, shadowing the generic type parameter. This may cause unexpected type inference and checking results.
(b) Accepted Answer: For clarity, we parse and check most TypeScript constructs even in JavaScript files. However, we are supposed to issue an error stating that "X is not valid in a JavaScript file," where in this case, X refers to type parameters.

code. Instead, it incorrectly allows the creation of a new type variable that shadows the original, potentially leading to type inference errors. The response generated by BugMentor highlights the need for better error reporting to prevent the misuse of TypeScript features in JavaScript code. This aligns with the accepted answer, which confirms that TypeScript should issue an error stating that "X is not valid in a JavaScript file," where "X" refers to type parameters in this case. Thus, BugMentor's answer not only provides additional context by explaining how TypeScript currently handles the type parameters but also helps the developer understand the root cause of the issue. With this information, the developer can now focus on reproducing and fixing the issue (e.g., misuse of TypeScript features in JavaScript code).

3. BugMentor : Proposed technique

Figs. 3, 4 and 6 show Steps 1, 2, 3 and 4 of the schematic diagram of our proposed technique — BugMentor. As input, it accepts a bug report of interest, its follow-up question, and a corpus of past bug reports with their follow-up questions and corresponding answers. As the output, our technique generates a relevant answer to the follow-up question. We discuss different steps of our technique in detail in the following sections.

3.1. Constructing the corpus

We construct our corpus using past bug reports and their discussion history from 20 real-world, open-source software systems on GitHub (Step 1, Fig. 3).

3.1.1. Choosing the repositories for the corpus

To construct our corpus, we collect high-quality repositories using a semi-automated approach. We follow the approach of Imran et al. (Imran et al., 2021) and choose GitHub as a data source. GitHub[5] is a popular open-source platform that supports various software maintenance practices, including bug report management. We select our repositories from GitHub as follows.

First, we select the most starred, active repositories containing a minimum of 500 issues (reported as of May 2023) using GitHub's advanced search https://github.com/search/advanced. The most active repositories with the most number of issues belonged to four programming languages — Python, Java, Javascript and C++. We then categorize the repositories into four subsets based on their programming languages, where each programming language had five repositories. The repositories that belonged to other programming languages that did not have atleast five repositories with the minimum number of issues were discarded.

3.1.2. Choosing the bug reports for the corpus

From each repository, we select the issues that were closed within the last five years to ensure the issues were recent and there has not been a concept drift. We also select the issues that are labeled as "bug", "crash", or "defect" to ensure that they are discussing software bugs or defects. We also select the bug reports labeled as "needs more info" and "stale", indicating that they were closed due to a lack of activity. We use GitHub's REST API[6] to collect the bug reports and their discussion history. Each of our collected bug reports consists of several fields, namely issue ID, title, bug description, bug reporter, label, creation time, and resolution time.

3.1.3. Selecting follow-up questions

To select follow-up questions from each bug report, we first collect their issue comments using a GitHub API client[7]. From each comment, we capture four different fields, namely comment ID, author of the comment, comment, and comment time. Following the strategy of Imran et al. (2021), we collect the comments that begin with an interrogative

[5] https://github.com/

[6] https://docs.github.com/en/rest?apiVersion=2022-11-28

[7] https://github-api.kohsuke.org/

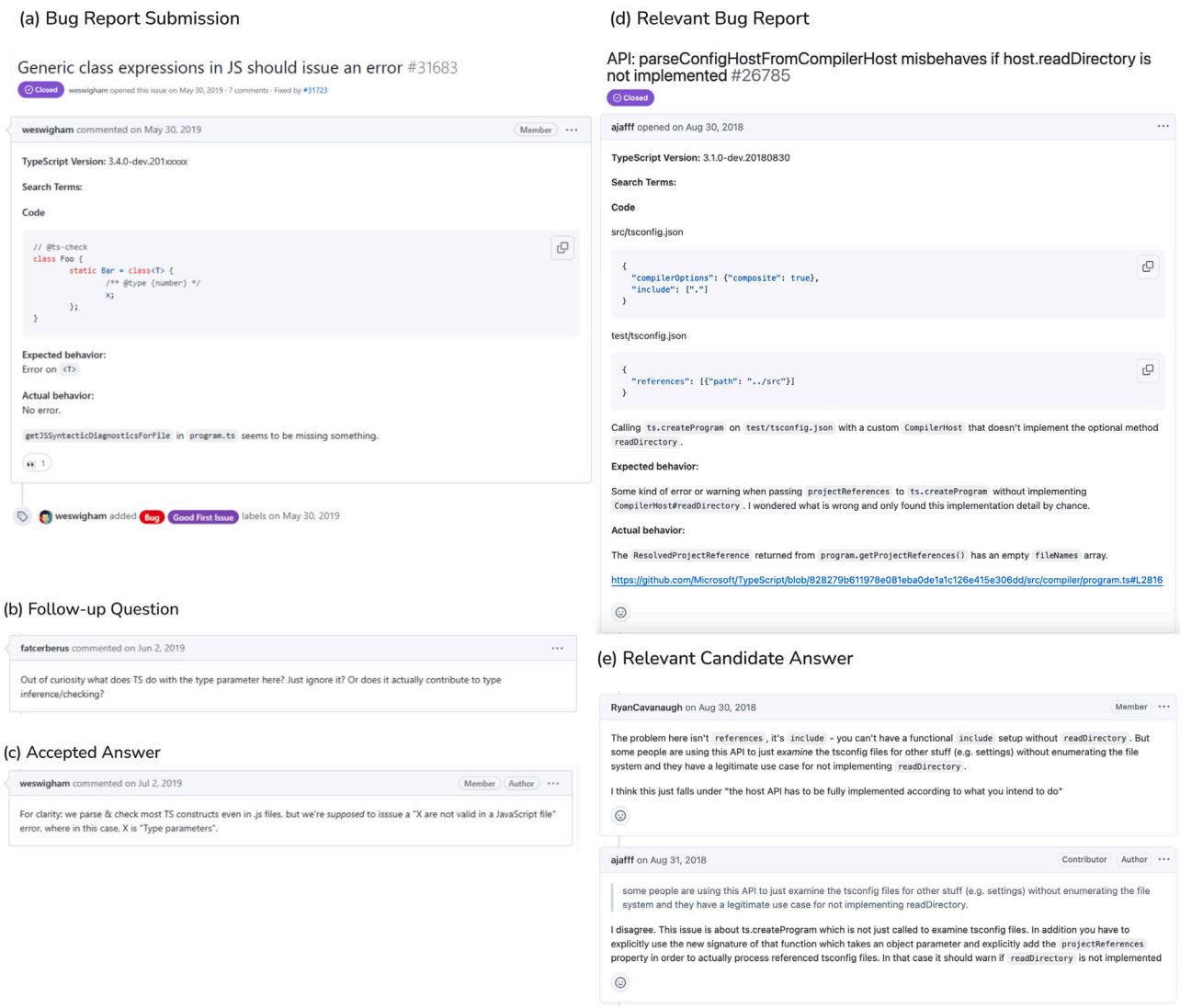

Fig. 2. An example bug report(ID #31683) with a follow-up question.

word and end with a question mark. We use NLTK's Classifier[8] to identify these comments, as was applied previously (Yuan et al., 2012). We also consider comments that requested additional information using words such as 'please' or 'can you' as valid comments. Then we select the first valid, interrogative comment that is not written by the bug reporter as our follow-up question from each bug report. We manually check up to 30 comments from each bug report to identify our follow-up questions.

3.1.4. Selecting candidate answers

The next step in our corpus construction is to select candidate answers against the follow-up questions above. We apply three criteria to the selection of our candidate answers: (a) Candidate Answer 1 — the first comment after the follow-up question that was not authored by the question submitter (Imran et al., 2021), (b) Candidate Answer 2 — the first comment after the follow-up question that was authored by the bug reporter, and (c) Candidate Answer 3 — the most similar comment to the follow-up question based on the BM25 algorithm (Whissell and Clarke, 2011) as the comment addressing the question would be textually similar to the question. Finally, our corpus consisted of hundreds of bug reports where we capture the Bug ID, title, description, follow-up question and three candidate answers from each bug report.

3.1.5. Data pre-processing

We apply standard natural language pre-processing to each bug report, follow-up question and candidate answer from our corpus. We remove redundant or noisy elements such as escape sequences, special characters, stack traces or images from each item (Imran et al., 2021) to facilitate better analysis. We retain all the stopwords and do not perform stemming to preserve the semantics of all the textual elements.

3.2. Capturing relevant answers

Once the corpus is constructed, we then capture relevant candidate answers against each follow-up question (Step 2, Fig. 4) and its respective project-specific corpus. We use the ElasticSearch[9] implementation of Lucene, a widely adopted search engine combining Boolean search and Vector Space Model (VSM) (Haiduc et al., 2013; Moreno et al., 2015; McCandless et al., 2010), for our task. We employ the Okapi

[8] https://www.nltk.org/_modules/nltk/classify

[9] https://www.elastic.co/elasticsearch/

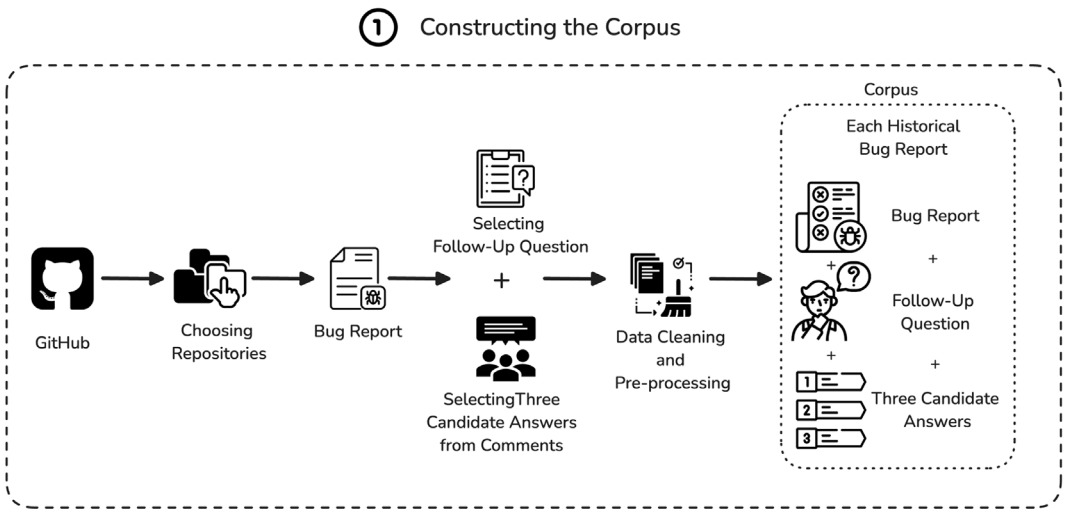

Fig. 3. Constructing the Corpus.

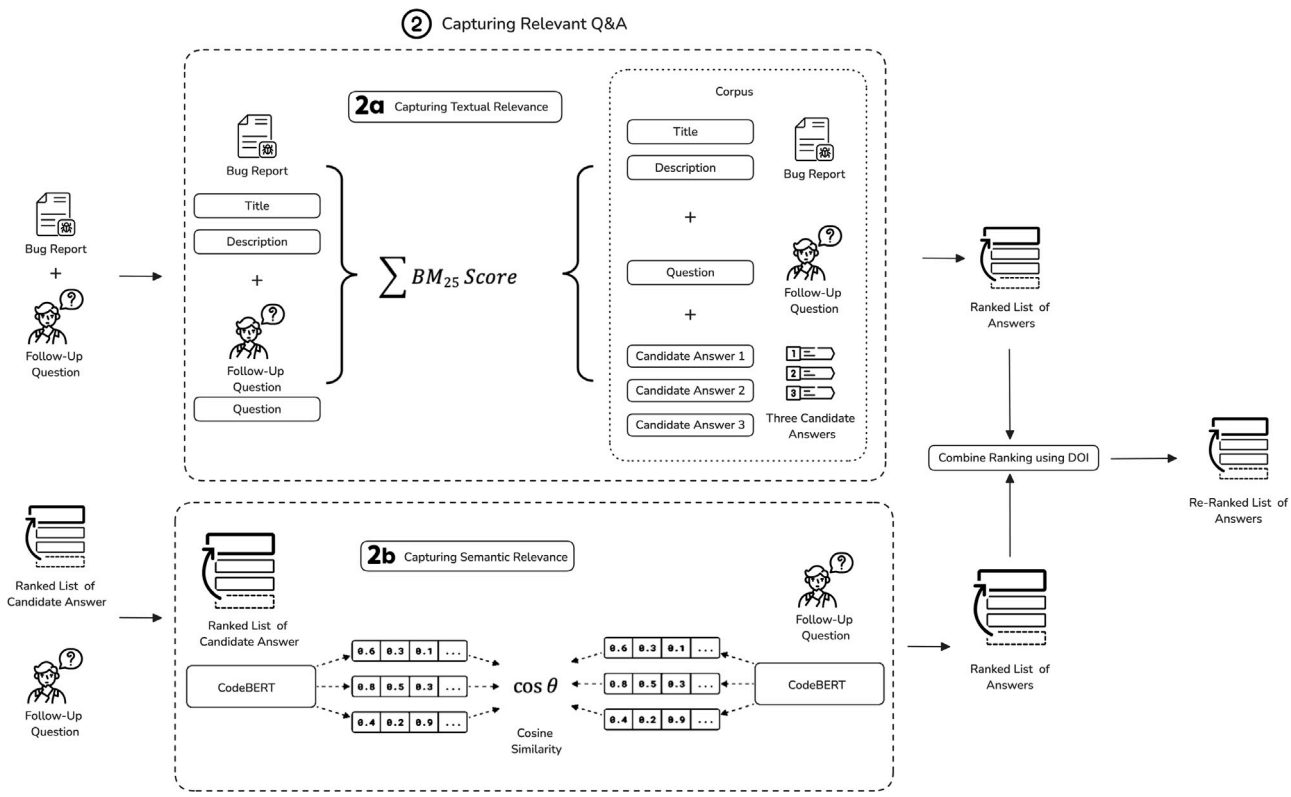

Fig. 4. Capturing relevant answers.

BM25 algorithm (Kamphuis et al., 2020) from the engine for similarity calculation. In particular, we calculate two BM25-based relevance scores where we adapt an existing work of Saha et al. (2013):

$$s'\left(\vec{d},\vec{q}\right) = \sum_{r\in Q}\sum_{f\in D} s(d_f, q_r) \tag{1}$$

Here q_r is a query representation, and d_f is a field from the past bug report (e.g., title, description).

3.2.1. Detecting relevant answers

First, we capture five items from each of the given and past bug reports — title, description, follow-up question, title + description, and title + description + question. Second, we capture five items from the given bug report — title, description, and follow-up question, title + description, and title + description + question and three candidate answers from each past bug report. Then, we conduct 5x8 similarity calculations using Eq. (1), for each of the items, as shown in Fig. 5. This score indicates the relevance between the follow-up question (and its bug report) and each candidate answer from the past bug report. According to existing literature (Saha et al., 2013), such an element-based similarity calculation can help prevent spurious matching. This score indicates the general relevance between a given and past bug reports. This score indicates the relevance between the follow-up question (and its bug report) and each candidate answer from the past bug report. The score from Step 2a indicates a more granular relevance between the question and answer. Thus, the above scores capture the relevance

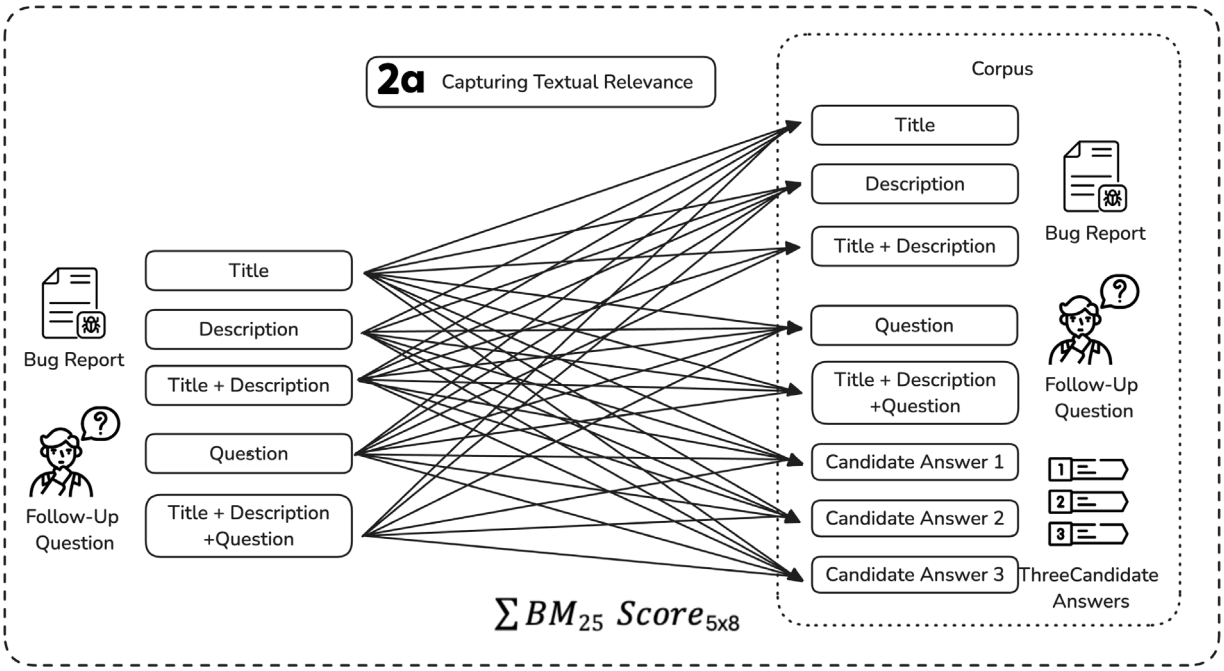

Fig. 5. Capturing relevant answers with structured information retrieval.

of each answer against the given follow-up question (and corresponding bug report).

3.2.2. Ranking based on textual relevance

We rank the candidate answers based on their BM25-based relevance scores calculated above (Step 2, Fig. 4). In particular, we capture the top K (e.g., 5) relevant candidate answers from the corpus against a follow-up question (Step 2, Fig. 4). It should be noted that these answers can come from various bug reports.

3.2.3. Ranking based on semantic relevance

BM25 algorithm relies on keyword matching for relevance estimation, which could suffer from the vocabulary mismatch problem (Furnas et al., 1987). We thus incorporate embedding-based similarity into our approach and detect the semantically relevant candidate answers. We capture word embeddings trained by CodeBERT (Feng et al., 2020) trained on CodeSearchNet (Husain et al., 2019). Candidate answers can contain both code and natural language elements. Since CodeBERT was fine-tuned for code and natural language tasks (Feng et al., 2020), it can represent our candidate answers better. We calculate the cosine similarity between the word embeddings of the follow-up question and that of each candidate answer. We then rerank the answers based on their semantic relevance to the question and return the top K answers (Step 2, Fig. 4).

3.2.4. Capturing the top relevant answers

We combine both BM25-based ranking and semantic relevance-based ranking using the Degree of Interest (DOI) method. Rahman et al. (2016) use the following formulae to combine two orthogonal rankings similar to both our rankings:

$$DOI = \frac{I}{N} \tag{2}$$

Here, I is the position of an answer in the ranked list and N is the total number of answers. First, we calculate the DOI score of each answer within the BM25-based list and then, we calculate the DOI score within the semantic relevance-based ranked list. Then we combined the DOI scores for each answer and rank the answers based on their combined DOI score using simple summation.

3.3. Answering the follow-up question

Once the relevant candidate answers are captured, we then construct the context and generate answers to the follow-up question using neural text generation (e.g Mistral).

3.3.1. Leveraging mistral for context-aware answer generation

To generate relevant answers to follow-up questions, we employ Mistral, a state-of-the-art pre-trained decoder-only Transformer model for neural text generation. Mistral has been pre-trained on internet-scale data and adopts an encoder–decoder network to generate texts (Jiang et al., 2023). We chose Mistral for our task due to its strong performance in context-aware text generation and its ability to handle long-range dependencies, which is crucial for processing bug report information (Jiang et al., 2023). From a performance-to-efficiency standpoint, Mistral offers a strong performance-to-efficiency ratio while remaining relatively lightweight compared to other modern language models, maintaining competitive performance despite its smaller size (Labrak et al., 2024). In terms of technical capabilities, Mistral's advanced attention mechanisms excel at processing multiple textual components like bug reports, descriptions, follow-up questions, and candidate answers, while supporting context-aware text generation (Jiang et al., 2023). Finally, from a deployment perspective, the model provides significant advantages through local deployment options without external API dependencies, flexibility for future fine-tuning, and efficient inference suitable for bug report analysis tasks (Wilkins and Rodriguez, 2024). While our technique remains model-agnostic and can incorporate other language models as needed, these combined factors made Mistral particularly suitable for generating accurate and contextually appropriate answers to follow-up questions in bug reports.

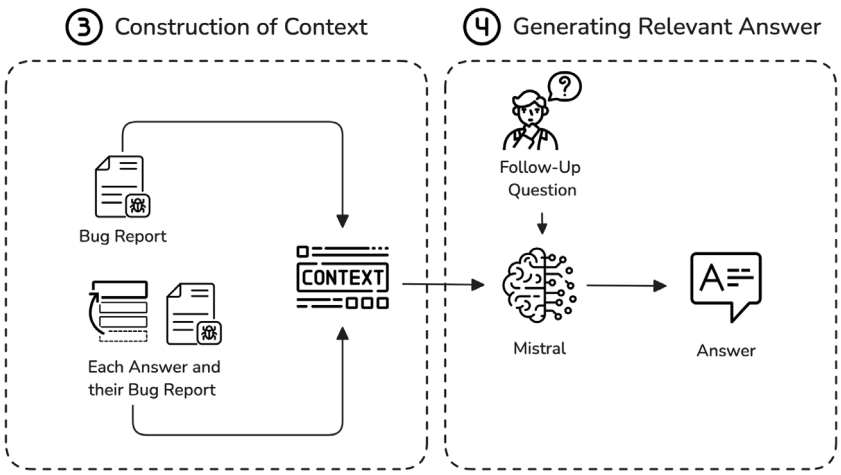

Fig. 6. Answering the follow-up question.

3.3.2. Constructing the context

Context plays a vital role in enabling generative question-answering models like Mistral to comprehensively understand the semantics and intent of a question, leading to more accurate and relevant answers[10] (Seonwoo et al., 2020). We then construct the context to enrich our follow-up question (Step 3, Fig. 6) using the relevant answers from the previous step. Thus, to construct the context, we use three items from the previous step — one answer from the ranked list, its bug report, and the given bug report. The answer and its bug report are likely to contain additional information to compensate for the missing information in the given bug report that triggers a follow-up question. The answer and its associated bug report likely contain additional details that can help address the follow-up question more comprehensively. We repeat the context construction for each of the K candidate answers and send them to our Mistral model for final answers.

3.3.3. Generating relevant answers

We then generate relevant answers to the follow-up question by leveraging our context above with Mistral (Step 4, Fig. 6). The model requires two components to operate, a question and its context. We provide the model with a follow-up question and its context from the previous step and capture an answer to the follow-up question from the model. We construct our prompt for our generation task following existing literature (Lazaridou et al., 2022; White et al., 2023) using chain of thought prompting (Mukherjee and Rahman, 2025). We designed a structured prompt to guide the language model in answering follow-up questions about bug reports. The prompt template is shown in Table 2. For example, our technique — BugMentor — offers the answer against the example question in Fig. 1 in Table 1.

Table 2
Prompt template.

Prompt Template
Answer Questions on the bug report based on the relevant information.
Here is a bug report which has incomplete information.
##Bug Report -
{bug_report}
There is a follow-up question asking for missing information.
Here is some relevant information from a previous bug report.
{relevant_candidate_answer}
Can you answer the question below based on the bug report?
Here is the question.
Question-
{question}
Answer :

4. Study methodology

We curated a large dataset containing 30,869 bug reports and their follow-up questions from 20 subject systems and evaluate BugMentor using four appropriate metrics from relevant literature — BLEU (Papineni et al., 2002), METEOR (Banerjee and Lavie, 2005), Semantic Similarity (Haque et al., 2022) and ROUGE (Lin, 2004). To place our work in the literature, we also conduct an ablation study (McCandless et al., 2010; Jiang et al., 2023) and compare our technique with three baseline techniques. Through our experiments, we answer four research questions as follows:

(a) **RQ$_1$**: How does BugMentor perform in answering follow-up questions in terms of different automatic evaluation metrics?
(b) **RQ$_2$**: Can BugMentor outperform the existing baselines in terms of automatic evaluation metrics?
(c) **RQ$_3$**: How do different components impact the overall performance of BugMentor?
(d) **RQ$_4$**: How does BugMentor perform in answering follow-up questions when compared with ChatGPT?
(e) **RQ$_5$**: How accurate, precise, useful, and concise are the answers from BugMentor?

4.1. Dataset construction

We first construct our corpus and ground truth for conducting the experiments.

4.1.1. Corpus construction

To construct our corpus, we chose the top 20 popular projects from GitHub written in 4 different programming languages and collected 30,869 bug reports from them. We also capture a follow-up question and three candidate answers from each bug report. Finally, our corpus consisted of hundreds of bug reports where we capture the Bug ID, title, description, follow-up question and three candidate answers from each bug report. We ensure that the candidate answers are selected from bug reports in the corpus submitted prior to the given bug report. We apply standard natural language pre-processing to each item from our corpus. Please check Section 3.1 for further details on corpus construction.

[10] https://huggingface.co/docs/transformers/tasks/question_answering

Table 3
Inter-annotator agreement.

Bucket 1		Bucket 2		Bucket 3		Bucket 4		Bucket 5		Bucket 6	
Pairs	κ	Pairs	κ	Pairs	κ	Pairs	κ	Pairs	κ	Pairs	κ
A1 & A2	0.72	A2 & A3	0.58	A3 & A4	0.44	A4 & A5	0.66	A5 & A6	0.49	A1 & A2	0.41
A2 & A3	0.24	A3 & A4	0.43	A4 & A5	0.53	A5 & A6	0.40	A1 & A5	0.44	A2 & A6	0.29
A3 & A1	0.39	A4 & A2	0.58	A5 & A3	0.65	A6 & A4	0.27	A6 & A1	0.49	A6 & A1	0.40

4.1.2. Ground-truth construction

To evaluate BugMentor, we first construct a randomly sampled, held-out dataset (i.e., 95% confidence level and 4.06% error margin) containing 550 bug reports (~27 bug reports x 20 systems). We then involve six human annotators (e.g., graduate students) to determine the ground truth answers against the follow-up questions from each bug report. The human annotators each have 2–3 years of experience, including 1–2 years of industry experience in the four programming languages used in our scenario. We divided 550 bug reports into six buckets (Table 3), each containing ~90 bug reports, their questions, and candidate answers. Each bucket was annotated by three annotators, resulting in ~270–275 bug reports per annotator. Then, we used majority voting (Kuhrmann et al., 2017) to determine the ground truth answers. That is, the answer having the majority of votes was chosen as the ground truth answer against a follow-up question. When the answers did not have a clear majority, i.e. for 3% of the dataset, the three annotators engaged in discussions to resolve conflicts and determine the ground truth answer (Kuhrmann et al., 2017). Each annotator spent ~2.5-3 h to complete the annotation task.

We compute the Cohen's κ for all pairs of annotators, and the result is reported in Table 3. Although we use majority voting for annotation, our calculated metrics show the agreement level for each pair of annotators. We found an average of 0.46, which indicates a moderate agreement between any two annotators.

4.2. Evaluation metrics

To evaluate BugMentor's answers against the ground truth, we use four relevant metrics from literature — BLEU Score (Papineni et al., 2002), METEOR Score (Banerjee and Lavie, 2005), ROUGE Score (Lin, 2004) and Semantic Similarity metric (Haque et al., 2022). They are defined as follows:

4.2.1. BLEU — Bi-Lingual Evaluation of Understudy:

BLEU score is a frequently used metric for evaluating translation (Papineni et al., 2002), which has found application in many software engineering tasks (e.g., comment generation (Hu et al., 2020), automatically explaining the bugs (Mahbub et al., 2023)). It compares a candidate text to a reference text and determines how similar they are based on the matching of their n-grams. The BLEU score is calculated as follows:

$$BLEU = BP \cdot exp\left(\sum_{n=1}^{N} w_n log(p_n)\right) \tag{3}$$

Here N is the maximum n-gram order, w_n is the weight assigned to the n-gram order, and BP is the brevity penalty — a factor that penalizes the BLEU score when the candidate text is shorter than the reference text. p_n is the modified n-gram precision, which measures the ratio of the overlapping n-grams (between the candidate text and the reference text), and the total number of n-grams in the candidate text.

4.2.2. METEOR — Metric for Evaluation of Translation with Explicit ORdering:

The METEOR score is a metric for evaluating the quality of machine translation output based on both lexical and syntactic information (Banerjee and Lavie, 2005). It measures the similarity between a candidate text and the reference text by sequentially applying exact match, stemmed match and wordnet-based synonym match between the texts. The METEOR score is computed as a combination of precision and recall of the matches, with a higher emphasis on recall. The formula for METEOR is given by:

$$METEOR = F_{mean} \cdot (1 - Penalty) \tag{4}$$

where F_{mean} is the harmonic mean of precision P and recall R, defined as:

$$F_{mean} = \frac{10 \cdot P \cdot R}{R + 9 \cdot P} \tag{5}$$

The penalty is a function of the chunkiness of the matched segments and is calculated as:

$$Penalty = 0.5 \cdot \left(\frac{\#\text{chunks}}{\#\text{unigrams in matches}}\right)^3 \tag{6}$$

This ensures that translations with fewer chunks of contiguous matched words are favored, reflecting higher order of correctness in terms of both semantics and syntax.

4.2.3. ROUGE — Recall-Oriented Understudy for Gisting Evaluation

ROUGE is a set of metrics designed to evaluate generated text to one or more reference texts (Lin, 2004). It measures the overlap of n-grams, word sequences, and word pairs between the candidate and reference texts. The most commonly used variants are ROUGE-N, ROUGE-L, and ROUGE-W. For our task, we use ROUGE-L, which measures the longest common subsequence (LCS) between the candidate and reference texts. ROUGE-L is more flexible as it does not require consecutive matches and can capture sentence-level word order effectively. The ROUGE-L score is calculated as follows:

$$R_{lcs} = \frac{LCS(X,Y)}{m}, \quad P_{lcs} = \frac{LCS(X,Y)}{n} \tag{7}$$

$$F_{lcs} = \frac{(1+\beta^2)R_{lcs}P_{lcs}}{R_{lcs} + \beta^2 P_{lcs}} \tag{8}$$

Where X is the reference summary of length m, Y is the candidate summary of length n, $LCS(X,Y)$ is the length of the longest common subsequence between X and Y, and β is usually set to favor recall ($\beta > 1$). ROUGE-L provides a comprehensive evaluation of summary quality by capturing both content overlap and sequence similarity, making it particularly suitable for our task of assessing generated text against reference texts.

4.2.4. SS — Semantic Similarity:

In a recent work, Haque et al. (2022) investigate which metric reflects human assessment of similarity the best. They use Sentence-BERT (Reimers and Gurevych, 2019) to capture meaningful sentence embeddings. Then they compare when a candidate text with the reference text based on these embeddings using cosine-similarity. According to Haque et al. such a similarity, it has the highest correlation with human-evaluated similarity. Semantic similarity can be computed as follows:

$$SemSim(ref, gen) = \cos(\text{sbert}(ref), \text{sbert}(gen)) \tag{9}$$

where $sbert(ref)$, $sbert(gen)$ are the numerical representations from Sentence-BERT for the reference text and generated text respectively.

4.3. Baseline selection

To the best of our knowledge, there exists no work that can offer relevant answers to follow-up questions from bug reports. However, there have been traditional methods and tools to support Q&A in several other contexts. Lucene (McCandless et al., 2010) is a popular IR-based tool that has been used to recommend answers in the programming Q&A site (Atwood, 2017; Ponzanelli et al., 2013). Mistral (Jiang et al., 2023) is a large language model for generating answers to general-purpose questions. AnswerBot (Xu et al., 2017) can synthesize answers for technical, non-factoid questions on StackOverflow. DialBERT (Li et al., 2020) can analyze mixed conversations using BERT and BiLSTM models and find comments that are most related to a given comment. Since our task also involves finding the most relevant answer to a given question, DialBERT could be a suitable baseline for our experiments. We thus consider these four as traditional method as baselines for our comparison. We call them Baseline_{Lucene}, $\text{Baseline}_{Mistral}$, AnswerBot, and DialBERT, respectively in this experiment.

To implement Baseline_{Lucene}, we provide a follow-up question as the *query* and all candidate answers as the *corpus* to the Lucene tool. Then we collect the top K answers from the tool by executing the query for our evaluation. For $\text{Baseline}_{Mistral}$, we provide a follow-up question as the *query* and its corresponding bug report as *context* to the Mistral model, which generates an answer. To implement DialBERT, we use the replication package provided by the authors (Li et al., 2020). We provide a follow-up question as the *query* and all bug reports along with their candidate answers as the *context*. To implement AnswerBot, we use the replication package provided by the authors (Xu et al., 2017; Bowen Xu, 2017). We provide a follow-up question as the *query* and all bug reports along with their candidate answers as the *corpus*.

5. Evaluation of BugMentor

5.1. Answering RQ_1 — How does BugMentor perform in answering follow-up questions in terms of different automatic evaluation metrics?

In this experiment, we analyze the performance of BugMentor using four different evaluation metrics — BLEU score (Papineni et al., 2002), Semantic Similarity (Haque et al., 2022), METEOR (Banerjee and Lavie, 2005) and ROUGE (Lin, 2004). We divide our held-out dataset into four subsets based on their corresponding programming languages and report the results for each subset. Table 4 shows the performance details of BugMentor. It should be noted that a higher value for BLEU, METEOR, Semantic Similarity, and ROUGE metrics is desirable in our experiments.

BugMentor achieves an average BLEU Score of 30.48 for Top 1 answer, and a maximum of 33.15 for Top 5 answers when our whole dataset is considered. However, our technique achieves a maximum of 34.38 for the Top 1, 34.99 for Top 3 and 36.18 for Top 5 answers across all four subsets. These BLEU scores indicate that our generated answers are *understandable* to *good* according to Google's AutoML Translation documentation[11]. They also show that the answers generated by BugMentor have a significant overlap with the ground truth in terms of their words and word order. However, BLEU score primarily focuses on determining the precision of an answer against the ground truth answer. Hence, we also evaluate our answers using the METEOR score, where recall is calculated by taking into account additional information such as synonyms, word forms, and sentence structures (Banerjee and Lavie, 2005).

As shown in Table 4, BugMentor achieves an average METEOR score of 0.69 for Top 1 answer, and a maximum of 0.75 for Top 5 answers against the whole dataset, which are considered to be reasonable (Agarwal and Lavie, 2008). Interestingly, it achieves a maximum of 0.79 for Top 1, 0.80 for Top 3, and 0.83 for Top 5 answers across four subsets. They show that BugMentor was able to produce a significant portion of the ground truth texts in the generated answers. However, since BLEU and METEOR scores rely on keyword matching between a generated answer and the ground truth answer, they may not capture the semantic relevance between them. Hence, we also evaluate our answers using ROUGE (Huang et al., 2016) and Semantic Similarity (Haque et al., 2022). They have also been shown to correlate better with human judgement of relevance (Papineni et al., 2002; Kusner et al., 2015).

In Table 4, we find that BugMentor achieves an average ROUGE of 0.46 for Top 1 answer, and a maximum of 0.50 for Top 5 answers, when the whole dataset is considered. It achieves a maximum of 0.48 for Top 1, 0.53 for Top 3, and 0.53 for Top 5 answers across four subsets. These scores show that BugMentor was able to generate answers semantically similar to the ground truth and were worded closely to the ground truth. However, ROUGE has limitations in capturing semantic nuances and contextual importance of words (Lin, 2004; Ng and Abrecht, 2015). It primarily measures surface-level text similarity and may miss valid paraphrases or alternative expressions that humans would recognize as equivalent. We thus also evaluate our answers using semantic similarity against the ground truth. The metric is appropriate when there may not be any syntactic overlap between the answers, which is a common phenomenon in question answering, according to existing literature (Yao et al., 2013; Severyn and Moschitti, 2015).

From Table 4, we also find that the answers from BugMentor have an average Semantic Similarity score of 62.04% for Top 1 answer, and a maximum of 67.30% for Top 5 answers when the whole dataset is considered. BugMentor achieves a maximum of 79.94% for Top 1, and a maximum of 82.35% for Top 5 answers when all subsets are considered. All these numbers indicate a high similarity in meaning and content between BugMentor's answers and the ground truth.

BugMentor's performance varies across programming languages. It consistently shows the best results across most metrics for the Java subset, particularly excelling in METEOR and Semantic Similarity. For the Python subset, BugMentor follows a close second, achieving high BLEU scores and demonstrating consistent strength in other metrics. However, BugMentor's performance for the JavaScript and C++ subsets is generally lower compared to its results for Java and Python subsets. This could be due to several factors. JavaScript's dynamic and loosely-typed nature may lead to more complex and ambiguous bug descriptions, making it harder to generate accurate answers (Jensen et al., 2009). C++'s low-level control and manual memory management may result in more intricate bugs that are difficult to address with our generated answers (Nilsen and Gao, 1995; Stroustrup, 1986).

Across all languages, BugMentor's performance improves when considering Top-3 and Top-5 results compared to Top-1, suggesting increased accuracy with more potential solutions. These patterns may be influenced by factors such as the complexity of the bugs, or the structure of the languages themselves. Languages like Java and Python, which have large, active communities, verbose and expressive syntax, and established ecosystems, may lead to more comprehensive and well-structured bug reports, resulting in better performance for BugMentor. On the other hand, languages with more dynamic and flexible features, like JavaScript (Jensen et al., 2009), or those with complex low-level control, like C++ (Nilsen and Gao, 1995; Stroustrup, 1986), may present challenges in generating accurate answers due to the potential ambiguity and complexity in bug descriptions. Considering these additional factors provides a more better understanding of their potential influences on BugMentor's performance across different programming languages.

We also extend our dataset from 550 samples (95% confidence level, 4.06% margin of error) to 1887 samples (98% confidence level, 2.5% margin of error). We repeat our experiment with the extended dataset and report the results. The improved sampling parameters and

[11] https://cloud.google.com/translate/automl/docs/evaluate

Table 4
Performance of BugMentor — 550 samples.

Metrics	Top K	Python	Java	JavaScript	C++	WholeDataset
BLEU ↑ (0–100)	Top-1	**34.38**	31.28	28.3	27.97	30.48
	Top-3	**34.99**	33.22	29.12	29.84	31.79
	Top-5	**35.25**	36.18	31.12	30.05	33.15
METEOR ↑ (0–1)	Top-1	0.68	**0.79**	0.66	0.62	0.69
	Top-3	0.72	**0.8**	0.71	0.65	0.72
	Top-5	0.76	**0.83**	0.73	0.69	0.75
SEMANTIC SIMILARITY ↑ (0–100)	Top-1	63.01	**79.94**	54.71	50.49	62.04
	Top-3	64.37	**80.25**	57.82	54.49	64.23
	Top-5	70.11	**82.35**	59.69	57.06	67.3
ROUGE ↑ (0–1)	Top-1	**0.48**	0.47	0.44	0.43	0.46
	Top-3	**0.53**	0.51	0.47	0.44	0.49
	Top-5	**0.53**	0.53	0.49	0.46	0.50

Table 5
Performance of BugMentor — 1887 samples.

Metrics	Top K	Python	Java	JavaScript	C++	Whole Dataset
BLEU ↑	Top-1	**66.50**	26.40	27.84	41.78	40.63
	Top-3	**72.08**	26.40	28.27	41.78	42.13
	Top-5	**72.08**	31.73	28.27	45.14	44.31
METEOR ↑	Top-1	**0.77**	0.47	0.40	0.74	0.60
	Top-3	**0.81**	0.47	0.43	0.75	0.61
	Top-5	**0.81**	0.52	0.43	0.75	0.63
SEMANTIC SIMILARITY ↑	Top-1	**91.00**	40.39	49.27	77.50	64.54
	Top-3	**92.50**	44.23	50.13	78.07	66.23
	Top-5	**92.50**	44.23	50.13	78.07	66.23
ROUGE ↑	Top-1	**0.72**	0.43	0.32	0.67	0.54
	Top-3	**0.77**	0.43	0.32	0.67	0.55
	Top-5	**0.77**	0.47	0.32	0.68	0.56

extended samples provide greater statistical power and better representation of diverse bug scenarios. Table 5 contains our results for the extended dataset.

We find performance improvements when 1887 samples were considered. BugMentor achieves an average BLEU Score of 40.63 for Top 1 answer, and a maximum of 44.31 for Top 5 answers when our whole dataset is considered. These BLEU scores indicate that our generated answers are of high quality according to Google's AutoML Translation documentation. BugMentor shows stronger results particularly in Python and C++, with Python achieving BLEU scores of 66.50 (Top-1) and C++ reaching 41.78 (Top-1), compared to their previous scores of 34.38 and 27.97 respectively. BugMentor's performance varies across programming languages. It consistently shows the best results across most metrics for the Python subset, excelling in BLEU and Semantic Similarity. For instance, Python achieves a Semantic Similarity score of 91.00 for Top-1 answer, significantly higher than other languages.

BugMentor's performance varies across programming languages when 1887 samples were considered. We find the best results across all metrics for the Python systems. For the C++ subset, it also achieves high Semantic Similarity and METEOR scores while demonstrating consistent strength in other metrics. However, BugMentor's performance measures for the Java and JavaScript subsets are consistently lower. This could happen due to several factors. First, JavaScript's dynamic, loosely-typed nature may lead to complex bugs and consequently ambiguous bug descriptions, making it harder for BugMentor to generate appropriate answers (Jensen et al., 2009). On the other hand, Java's verbose syntax, strong type system, and object-oriented paradigm could lead to bugs spanning multiple program components. This could also make it harder for BugMentor to provide helpful answers to follow-up questions, especially when those questions involve complex class interactions or rely on deeper knowledge of the broader Java ecosystem.

We also conducted a cross-project evaluation to assess the generalizability of BugMentor, especially in scenarios where a new project may have limited or no historical bug reports available for retrieval. In our experiment, a target bug report was drawn from one project in a given programming language, and the retrieval corpus (i.e., the candidate pool) was composed of bug reports from other projects in the same programming language. We repeat this for every project in each of the four programming languages and evaluate BugMentor's performance. We also compared it with three baselines — ChatGPT, Lucene, and AnswerBot — with the same setting.

Our results from these cross-project experiments are shown in Table 6. We see that BugMentor consistently outperforms all baselines across languages and metrics, despite no access to in-project historical data. However, the performance varies across the programming languages. For example, in Python, BugMentor delivers more accurate answers than the baselines, achieving a Top-1 BLEU score of 58.44, which is higher than those of ChatGPT (11.42), Lucene (5.38) and AnswerBot (6.93). We observe similar trends in the other programming languages. For C++, BugMentor achieves a Top-1 BLEU score of 34.44 compared to 7.35 of ChatGPT, and its best Semantic Similarity of 70.28 (at Top-3) compared to ChatGPT of 33.52 (at Top-5). Although BugMentor achieves lower cross-project scores in Java and JavaScript compared to Python, it still consistently outperforms the baselines in those languages. For example, in Java, BugMentor's Top-5 BLEU is 27.74 compared to 8.38 of ChatGPT, and in JavaScript. The Semantic Similarity of BugMentor peaks at 44.95 (Top-3) compared to 29.11 of ChatGPT. These findings indicate that BugMentor is capable of working with projects having limited or no bug reporting history, making it feasible for adoption in new or low-resource software systems.

Summary of RQ_1: BugMentor achieves an average BLEU score of 44, which shows that it can generate relevant answers to follow-up questions that are *high quality* according to Google's Standard. It achieves an average semantic similarity score of 67%, which shows that the answers also have a high semantic overlap with the ground truth answers. We also find that BugMentor performs the best for Python across most metrics while suffering from low performance with projects written in Java, JavaScript and C++.

5.2. *Answering RQ_2 — Can BugMentor outperform the existing baselines in terms of automatic evaluation metrics?*

In this experiment, we evaluate the answers from all four baselines — Lucene (McCandless et al., 2010), Mistral (Jiang et al., 2023), AnswerBot (Xu et al., 2017), and DialBERT (Li et al., 2020) — against the ground truth using four evaluation metrics. Tables 7 and 8 show the comparison details between BugMentor and these baseline techniques on 550 and 1887 samples, respectively.

BugMentor outperforms Baseline_{Lucene} across all four metrics on the 550 dataset. BugMentor achieves 43%–69% performance improvement in terms of BLEU, 14%–17% in terms of METEOR, 26%–28%

Table 6
Performance of BugMentor in cross-project setting.

Metrics	Top K	Lucene				AnswerBot				ChatGPT				BugMentor			
		Python	Java	JavaScript	C++	Python	Java	JavaScript	C++	Python	Java	JavaScript	C++	Python	Java	JavaScript	C++
BLEU ↑	Top-1	5.38	6.07	0.70	7.20	6.93	6.77	5.40	2.83	11.42	7.70	13.09	7.35	58.44	22.50	24.23	34.44
	Top-3	5.38	6.07	0.70	7.20	7.96	7.10	5.40	6.10	11.42	8.02	13.09	7.53	62.70	25.50	26.94	36.37
	Top-5	6.46	6.07	0.70	7.20	7.96	7.10	6.34	8.89	13.7	8.38	15.71	7.78	63.88	27.74	26.18	37.35
METEOR ↑	Top-1	0.20	0.06	0.14	0.13	0.27	0.24	0.21	0.26	0.36	0.32	0.35	0.30	0.68	0.41	0.34	0.60
	Top-3	0.21	0.06	0.14	0.13	0.28	0.24	0.22	0.27	0.36	0.32	0.35	0.30	0.70	0.43	0.36	0.62
	Top-5	0.24	0.06	0.14	0.13	0.29	0.25	0.23	0.27	0.39	0.32	0.37	0.31	0.71	0.45	0.36	0.63
SEMANTIC SIMILARITY ↑	Top-1	28.97	23.33	25.76	25.79	21.33	24.31	21.31	23.01	40.32	38.17	27.47	32.61	81.92	38.82	42.46	68.03
	Top-3	29.77	23.75	26.51	26.66	23.47	24.45	21.34	23.58	48.61	38.27	27.78	32.73	81.92	41.05	44.95	70.28
	Top-5	34.05	25.48	30.44	31.61	25.65	24.45	22.45	23.89	45.98	39.17	29.11	33.52	82.58	41.49	44.58	70.28
ROUGE ↑	Top-1	0.36	0.03	0.24	0.25	0.31	0.26	0.22	0.28	0.37	0.34	0.35	0.33	0.61	0.39	0.30	0.55
	Top-3	0.36	0.03	0.24	0.25	0.31	0.26	0.23	0.29	0.37	0.35	0.35	0.33	0.65	0.41	0.32	0.56
	Top-5	0.42	0.03	0.24	0.25	0.33	0.27	0.25	0.31	0.40	0.35	0.37	0.34	0.66	0.42	0.32	0.57

Table 7
Comparison of BugMentor with Baseline$_{Lucene}$, DialBERT, AnswerBot and Baseline$_{Mistral}$ — 550 samples.

Metrics	Top K	Baseline$_{Lucene}$	DialBERT	AnswerBot	Baseline$_{Mistral}$	BugMentor
BLEU ↑	Top-1	18.02	9.85	4.31	20.69	30.48
	Top-3	20.18	12.55	6.71		31.79
	Top-5	23.04	12.78	6.71		33.15
METEOR ↑	Top-1	0.59	0.31	0.05	0.59	0.69
	Top-3	0.63	0.34	0.12		0.72
	Top-5	0.64	0.38	0.12		0.75
SEMANTIC SIMILARITY ↑	Top-1	48.18	61.38	36.47	57.02	62.04
	Top-3	50.88	64.16	47.69		64.23
	Top-5	52.92	64.88	47.69		67.30
ROUGE ↑	Top-1	0.43	0.43	0.16	0.42	0.46
	Top-3	0.45	0.46	0.17		0.49
	Top-5	0.45	0.46	0.19		0.50

in terms of Semantic Similarity and 6%–11% in terms of ROUGE. Baseline$_{Mistral}$ also performs poorly compared to both BugMentor. BugMentor achieves a performance improvement of 47% in terms of BLEU, 16% in terms of METEOR, 8% in terms of Semantic Similarity and 9% in terms of ROUGE. When compared with DialBERT, BugMentor achieves 153%–209% performance improvement for BLEU, 97%–122% in terms of METEOR, 1%–3% in terms of Semantic Similarity and 6%–8% in terms of ROUGE. Similarly, Answerbot performs poorly when compared to BugMentor. BugMentor achieves a performance improvement of 373%–607% in terms of BLEU, 500%–1200% in terms of METEOR, 34%–70% in terms of Semantic Similarity and 163%- 187% in terms of ROUGE.

BugMentor also outperforms all baseline techniques across all four metrics, even with the extended dataset. Baseline$_{Mistral}$ and Baseline$_{Lucene}$ are the second closest in performance. Our proposed technique — BugMentor — outperforms Baseline$_{Mistral}$ by 49.65% higher BLEU, 9.09% higher METEOR, 54.03% higher Semantic Similarity and 5.88% higher ROGUE. On the other hand, the performance improvements over Baseline$_{Lucene}$ are 78%–91% in terms of BLEU, 40%–46% in terms of METEOR, 24%–30% in terms of Semantic Similarity and 19%–22% in terms of ROUGE. When compared with DialBERT, BugMentor achieves 100%–120% performance improvement for BLEU, 26%–33% in terms of METEOR, 3%–7% in terms of Semantic Similarity and 9%–14% in terms of ROUGE. Similarly, BugMentor outperforms AnswerBot by a large margin.

We also perform Mann–Whitney Wilcoxon test (Cuzick, 1985) to check if the performance of Baseline$_{Lucene}$, Baseline$_{Mistral}$, DialBERT and AnswerBot are significantly lower than that of BugMentor using Bonferroni Correction (Weisstein, 2004) on both 550 and 1887 datasets. We find that BugMentor performs significantly higher than Baseline$_{Lucene}$, AnswerBot and DialBERT, i.e., p-value = 0.00016, 00016 and 0.0016 <0.016 in terms of all four metrics for 550 dataset and p-value = 0.00016, 0.0016 and 0.00016 < 0.016 in terms of all four metrics for the 1887 dataset. To quantify the magnitude of the differences, we calculate Cliff's Delta effect sizes (Cohen, 1988). The effect sizes for BugMentor compared to Baseline$_{Lucene}$, AnswerBot, and DialBERT are 0.25, 0.38 and 0.32, for the 550 dataset and 0.38, 0.50 and 0.38 for the 1887 dataset, indicating small to medium effect.

> **Summary of RQ$_2$:** BugMentor performs better in answer generation than all four baselines in terms of four evaluation metrics. According to the statistical significance test, BugMentor outperforms the closest competitors Baseline$_{Lucene}$, DialBERT and AnswerBot with statistical significance and small to medium effect.

5.3. Answering RQ$_3$ — How do different components impact the overall performance of BugMentor?

To answers a follow-up question our technique combines three key components — (a) structured information retrieval, (b) embedding similarity-based ranking and (c) neural text generation. In this experiment, we conduct an ablation study to determine the contribution of each component. In particular, we design different variants of BugMentor with each component and evaluate their performance in answer generation. Table 9 summarizes our results from the ablation study.

We find that BugMentor outperforms all of its variants based on either individual components or their combinations. The combination of structured information retrieval and embedding similarity-based ranking (a.k.a., BugMentor$_{L+E}$) is a close second when compared using BLEU and ROUGE scores.

From Table 9, we find that the combination of Embedding and Mistral is the closest competitor to BugMentor. However, the absence of the text generation component (a.k.a., Mistral) and structured information retrieval component (a.k.a Lucene) reduces the performance from that of BugMentor by 44.45% in terms of BLEU, 50% in terms of

Table 8
Comparison of BugMentor with Baseline_{Lucene}, DialBERT, AnswerBot and $\text{Baseline}_{Mistral}$ — 1887 samples.

Metrics	Top K	Baseline_Lucene	DialBERT	AnswerBot	Baseline_Mistral	BugMentor
BLEU ↑	Top-1	21.25	18.40	14.40	27.15	**40.63**
	Top-3	23.34	20.80	15.66		**42.13**
	Top-5	24.85	22.10	16.21		**44.31**
METEOR ↑	Top-1	0.41	0.45	0.19	0.55	**0.60**
	Top-3	0.43	0.48	0.19		**0.61**
	Top-5	0.45	0.50	0.21		**0.63**
SEMANTIC SIMILARITY ↑	Top-1	49.50	59.80	23.38	41.90	**64.54**
	Top-3	51.75	63.10	25.46		**66.23**
	Top-5	53.00	64.25	26.05		**66.23**
ROUGE ↑	Top-1	0.44	0.47	0.18	0.51	**0.54**
	Top-3	0.46	0.49	0.19		**0.55**
	Top-5	0.47	0.51	0.20		**0.56**

Table 9
Experimental results from the ablation study.

Metrics	Top K	BugMentor_{L}	BugMentor_{L+E}	BugMentor_{L+M}	BugMentor_{M}	BugMentor_{E}	BugMentor_{E+M}	BugMentor_{RO+M}	BugMentor
BLEU ↑	Top-1	21.25	24.93	8.68	20.69	21.1	25.2	10.98	30.48
	Top-3	23.47	26.63	9.33		23.3	26.9	10.98	31.79
	Top-5	24.48	29.35	9.79		24.7	28.3	11.18	33.15
METEOR ↑	Top-1	0.31	0.52	0.41	0.59	0.46	0.58	0.35	0.69
	Top-3	0.33	0.55	0.43		0.48	0.61	0.35	0.72
	Top-5	0.35	0.57	0.44		0.5	0.63	0.36	0.75
SEMANTIC SIMILARITY ↑	Top-1	48.5	55.39	33.56	73.61	55.2	61.9	27.71	62.04
	Top-3	50.98	56.3	35.15		57.5	62.6	27.84	64.23
	Top-5	53.51	57.02	35.84		59.8	65.01	27.89	67.3
ROUGE ↑	Top-1	0.41	0.44	0.34	0.42	0.4	0.44	0.35	0.46
	Top-3	0.45	0.47	0.36		0.43	0.46	0.35	0.49
	Top-5	0.46	0.49	0.36		0.44	0.47	0.35	0.5

BugMentor_{L} = $\text{BugMentor}_{Lucene}$, BugMentor_{E} = $\text{BugMentor}_{Embedding}$, BugMentor_{M} = $\text{BugMentor}_{Mistral}$, BugMentor_{L+E} = $\text{BugMentor}_{Lucene+Embedding}$, BugMentor_{L+M} = $\text{BugMentor}_{Lucene+Mistral}$, BugMentor_{E+M} = $\text{BugMentor}_{Embedding+Mistral}$, BugMentor_{RO+M} = $\text{BugMentor}_{RetrievalOnly+Mistral}$

METEOR, 12.39% in terms of Semantic Similarity and 15% in terms of ROUGE. We also note that the combination of Lucene and Embedding (a.k.a., BugMentor_{L+E}) is the second closest competitor to BugMentor when BLEU is considered. However, the absence of the text generation component (a.k.a., Mistral) reduces the performance from that of BugMentor by 18.20% in terms of BLEU and 10.71% in terms of Semantic Similarity when the Top 1 answer is captured and 4.08% in terms of ROUGE when Top 3 is captured. In other words, the text generation component improves the semantic closeness between the generated answers and the ground truth answer. Thus, the Mistral component has a significant impact on improving not only the understandability of our generated answers (i.e., BLEU) but also their syntactic and semantic relevance to the ground truth answers (i.e. METEOR, ROUGE, Semantic Similarity).

We also include a retrieval-only prompting baseline, BugMentor_{RO+M}, which directly supplies the top-ranked historical Q&A pair to the LLM without any structured reasoning or context integration. Theoretically, this lightweight strategy serves as a lower bound for our approach. As expected, this baseline (i.e., BugMentor_{RO+M}) performs significantly worse than the full BugMentor. For instance, BugMentor_{RO+M} achieves a lower BLEU of 10.98 for the Top-1 result compared to 30.48 for BugMentor. Similarly, inferior METEOR and Semantic Similarity metrics were observed for the baseline (Table 9). This pronounced gap in performance confirms that simply prompting the LLM with a single retrieved Q&A pair is insufficient, underscoring the importance of BugMentor's combination of structured retrieval, ranking, and context-enhanced generation for robust performance.

From Table 9, we also find that the components from BugMentor do not perform well when evaluated individually. For example, if we omit both Embedding and Mistral components and only use Lucene, the performance degrades by 30.28% in terms of BLEU, 55.07% in terms of METEOR, 21.82% in terms of Semantic Similarity and 10.86% in terms of ROUGE for the Top 1 answer. Similarly, if we use only the Mistral component, the performance degrades by 32.11% in terms of BLEU, 14.49% in terms of METEOR and 8.09% in terms of Semantic Similarity and 8.69% in terms of ROUGE for Top 1 answer. Thus, these components in isolation might not be sufficient to deliver appropriate answers to the follow-up questions from bug reports.

We also investigated BugMentor's compatibility with different backbone LLMs. In particular, we replaced the Mistral component with the Llama variant and repeated our experiments. Table 10 summarizes our results.

Although we initially selected Mistral for the neural text generation component due to its practical advantages (e.g., lower memory consumption and faster inference compared to Llama (Mistral AI team, 2023)), we extended our experiments to compare with Llama. Specifically, we evaluated $\text{Baseline}Mistral$, $\text{Baseline}Llama$, $\text{BugMentor}Llama$, and $\text{Baseline}Mistral$ on the extended dataset. As shown in Table 1, both Baseline models perform poorly in isolation, but the addition of BugMentor's retrieval and ranking context yields substantial improvements. BugMentor improves BLEU scores by 49.65% over standalone Mistral, while $\text{BugMentor}Llama$ improves by 33.73% over standalone Llama. Importantly, $\text{BugMentor}Llama$ achieves higher absolute performance across BLEU, METEOR, and Semantic Similarity than $\text{BugMentor}Mistral$, highlighting Llama's strong potential for bug-related answer generation.

Beyond accuracy, deployment cost was an important consideration for us. Mistral's lower memory footprint and faster inference make it well-suited for resource-constrained environments (e.g., edge devices or lightweight IDEs). In contrast, Llama, being larger by design, requires greater computational resources, yet consistently delivers stronger empirical performance across all metrics. These findings highlight that BugMentor is compatible with both models — Llama offers superior accuracy when resources permit, while Mistral provides a cost-effective option for efficiency-critical deployments.

Table 10
Comparison of BugMentor with Baseline_{LLama}, BugMentor_{LLama} and $\text{Baseline}_{Mistral}$.

Metrics	Top K	BugMentor_{LLama}	BugMentor	Baseline_{LLama}	$\text{Baseline}_{Mistral}$
BLEU ↑	Top-1	52.8	40.63	30.38	27.15
	Top-3	54.35	42.13		
	Top-5	56.1	44.31		
METEOR ↑	Top-1	0.64	0.6	0.49	0.55
	Top-3	0.66	0.61		
	Top-5	0.68	0.63		
SEMANTIC SIMILARITY ↑	Top-1	71.1	64.54	67.91	61.9
	Top-3	72.5	66.23		
	Top-5	72.5	66.23		
ROUGE ↑	Top-1	0.58	0.54	0.48	0.51
	Top-3	0.58	0.55		
	Top-5	0.62	0.56		

Summary of RQ_3: Our ablation study demonstrates significant contribution of the three components — structured information retrieval, embedding similarity-based ranking, and neural text generation (a.k.a., LLM backbone) — towards the overall performance of BugMentor. We also found that BugMentor outperforms its five variants based on individual components, their combinations, or even a retrieval-only baseline, which justifies the presence of all three components in BugMentor. We also found that BugMentor is model-agnostic and can deliver superior performance in question answering with different LLM backbones (e.g., Mistral, Llama).

5.4. Answering RQ_4: how does BugMentor perform in answering follow-up questions when compared with chatgpt?

Besides the traditional baselines, it is important to consider the emerging, popular LLM-based approaches such as ChatGPT for a comparative analysis. Since the most recent model of ChatGPT i.e., GPT-4o is a paid model, we conduct a quantitative analysis using ChatGPT. We use 550 bug reports ∼27 bug reports x 20 systems) for our analysis. First, we collect the bug reports, follow-up questions, ground truth answers and the answers generated by BugMentor. Then, we provide each follow-up question and its corresponding bug report in the same prompt and collect the answers from ChatGPT for our comparison. Tables 11 and 12 summarize our comparison details. Additionally, we replace the Mistral component of BugMentor with the ChatGPT component and repeat our experiments. We conduct experiments to determine the performance when past bug reports are provided by BugMentor for contextual information.

From Table 11, we observe that ChatGPT is able to achieve a BLEU score of 13.22 which indicates that the answers from ChatGPT does not have a significant overlap with the ground truth text. We also find the METEOR score, since it takes into account synonyms, word forms and sentence structures. It achieves a METEOR score of 0.34 which shows that the answers have a poor keyword overlap. We also evaluate ChatGPT's answers using Semantic Similarity and ROUGE scores which better captures semantic and contextual importance of words. We observe that the answers from ChatGPT achieve a Semantic Similarity of 43.99 and ROUGE score of 0.41 which indicates a low semantic overlap with the ground truth answers. In contrast, BugMentor demonstrates a significant performance improvement across all metrics. Compared to ChatGPT, BugMentor achieves a 130.5% improvement in terms of BLEU score, a 102.9% improvement in METEOR score, a 41.03% improvement in Semantic Similarity, and a 12% improvement in ROUGE score. These results highlight BugMentor's superior ability to generate answers that closely resemble the ground truth, both in terms of syntactic and semantic similarity.

Table 11
Comparison between $\text{BugMentor}_{ChatGPT}$, BugMentor and ChatGPT — 550 samples.

Metrics	Top K	$\text{BugMentor}_{ChatGPT}$	BugMentor	ChatGPT
BLEU ↑	Top-1	52.35	30.48	20.09
	Top-3	54.72	31.79	
	Top-5	56.89	33.15	
METEOR ↑	Top-1	0.753	0.69	0.46
	Top-3	0.781	0.72	
	Top-5	0.802	0.75	
SEMANTIC SIMILARITY ↑	Top-1	82.45	62.04	65.64
	Top-3	85.12	64.23	
	Top-5	88.37	67.3	
ROUGE ↑	Top-1	0.502	0.46	0.48
	Top-3	0.531	0.49	
	Top-5	0.553	0.50	

Table 12
Comparison between $\text{BugMentor}_{ChatGPT}$, BugMentor and ChatGPT — 1887 samples.

Metrics	Top K	$\text{BugMentor}_{ChatGPT}$	BugMentor	ChatGPT
BLEU ↑	Top-1	36.03	**40.63**	28.57
	Top-3	38.59	**42.13**	
	Top-5	**44.8**	44.31	
METEOR ↑	Top-1	0.46	**0.6**	0.51
	Top-3	0.49	**0.61**	
	Top-5	0.53	**0.63**	
SEMANTIC SIMILARITY ↑	Top-1	**69.79**	64.54	57.75
	Top-3	**71.74**	66.23	
	Top-5	**77.42**	66.23	
ROUGE ↑	Top-1	**0.56**	0.54	0.54
	Top-3	**0.58**	0.55	
	Top-5	**0.61**	0.56	

We compare the performance of three systems — $\text{BugMentor}_{ChatGPT}$, BugMentor, and ChatGPT — across four metrics for various Top-K answers for both 550 and 1887 datasets (Tables 11 and 12). $\text{BugMentor}_{ChatGPT}$ consistently outperforms the other two systems across all metrics and Top-K answers for the 550 datasets. For example, in the BLEU metric, which measures text accuracy and fluency, $\text{BugMentor}_{ChatGPT}$ achieves a score of 52.35 at Top-1, significantly higher than BugMentor (30.48) and ChatGPT (20.09). This trend continues at Top-3 and Top-5, demonstrating the model's ability to generate more accurate answers consistently. Similarly, in the METEOR metric, which evaluates text quality based on precision, recall, and alignment, $\text{BugMentor}_{ChatGPT}$ scores 0.802 at Top-5, compared to 0.75 for BugMentor, indicating better semantic and syntactic alignment with reference texts. This suggests that $\text{BugMentor}_{ChatGPT}$ produces

Table 13
Quality aspects of the generated answers.

Quality	Overview
Accurate	It provides the same factual information as the reference.
Precise	It can answer the question completely
Concise	It is short and still answers the question.
Useful	The provided information has the potential to answer the question.

responses that are not only more grammatically and syntactically accurate but also semantically richer and closer to the intended meaning. In the ROUGE metric, which measures the overlap of n-grams between generated and reference texts, BugMentor$_{ChatGPT}$ also performs better, scoring 0.553 at Top-5 compared to 0.50 for BugMentor. This indicates that BugMentor$_{ChatGPT}$ generates more relevant and precise text.

We compare the performance of three systems for the 1887 datasets. We find that BugMentor consistently outperforms the other two methods across BLEU and METEOR. For example, in terms of METEOR, BugMentor outperforms BugMentor$_{ChatGPT}$ by 18%–30% for Top 1,3 and 5 results. Similarly, in terms of BLEU, BugMentor outperforms BugMentor$_{ChatGPT}$ by 9%–12% for Top 1 and Top 3. This suggests that BugMentor generates answers that are syntactically accurate, semantically richer and closer to the ground truth texts. However, we find BugMentor$_{ChatGPT}$ performs marginally better in the Top 5 results of BLEU by 1%. However, we observe that BugMentor$_{ChatGPT}$ outperforms BugMentor in terms of Semantic Similarity by 7%–14% and ROUGE by 3%–8%. Thus, BugMentor provides more accurate responses that align closely with reference solutions, while BugMentor$_{ChatGPT}$ covers a wider range of semantic variations. As shown above, our approach can work with various language models, indicating its flexibility and modularity.

To demonstrate the benefit of BugMentor, we present a use case (Bug Report #1182) and compare with ChatGPT head to head below. The bug report discusses a performance issue with image detection in TorchServe, where inference times were significantly impacted by network delays when transferring large images. Despite being on the same network as TorchServe, the reporter experienced transfer times of 2–3 min before pre-processing could begin, compared to the actual inference time of 10–20 s. However, the bug report does not provide sufficient details. A developer asks more details about the setup and how the transfer time versus the pre-processing time are measured and if TorchServe is running on the same machine, where the images are located. The ground truth reveals that TorchServe runs on the same machine as the images, with data transferred over localhost. The reporter measures time by logging timestamps at various points, finding a 2 min delay between sending the request and the start of pre-processing. The reporter also provides their machine specifications. From the below example we can see that BugMentor's Answer is more aligned with the ground truth, and suggests logging at different stages, capturing the context of the bug report. Additionally, BugMentor's answer provides additional context about the performance differences between WSL and Docker that are not mentioned in the ground truth. In comparison, ChatGPT is also able to capture the context of transfer and pre-processing time and suggests logging, however, it does not provide additional information about the performance difference between Docker on Windows and WSL.

Summary of RQ$_4$: BugMentor outperforms ChatGPT in answering follow-up questions for a sample of 228 bug reports across four languages. While ChatGPT understands bug context, it lacks specificity in its answers. BugMentor, by learning from historical bug data, provides more accurate and contextual responses, surpassing ChatGPT on metrics like BLEU, METEOR, Semantic Similarity, and ROUGE scores.

Bug #1182:
Title: Slow Connections with Image Detection
Description: Is your feature request related to a problem? Please describe: Inference times with image detection seem to be slowed down by network delays with large images. I'm on the same network as TorchServe, but transferring large images to TorchServe takes about 2–3 min before preprocessing even starts, compared to 10–20 s for inference. I could move the preprocessing outside of TorchServe, but that would defeat the purpose of having all model-related code contained by TorchServe. Even then, my preprocessing only resizes the images.

Describe the solution: Can the code include strategies or options to reduce network transfer times of images, possibly by changing the connection type? Describe alternative solutions: The documentation could also be updated to show how to mitigate this issue outside of TorchServe.

Question: Interesting, I think this would be valuable for us to repro if you have anything we could use. Is torchserve running on the same machine as where your images are? Any details you can provide would be super helpful How are you measuring the transfer time vs preprocess time?

Ground Truth: Torchserve is running on the same machine as where my images are (and on the same disk!). So all data is being transferred over localhost connections. In my torchserve handler, I'm printing the time as soon as it hits the'preprocess()'function. Then similar time prints allow me to measure the time spent on'preprocess()','inference()', and'postprocess()'.

The main outcome here is that the time from just before I send the request to the'preprocess()'function starting tends to be about 2 min on average.

I'm using an efficientdet d4 object detection model, but I don't think the type of model should matter based on this information?

My machine specs are windows10 (running in WSL), Ryzen 7 3700x, 16 GB RAM, GTX1060 6 GB

Bugmentor Answer TorchServe is running on the same machine as the images, with data transferred over localhost. To measure the transfer delay, the time is logged before sending the image and when preprocessing starts. The observed delay is around 2 min. Notably, this issue doesn't occur in WSL 2 but does in Windows, suggesting a performance difference between Docker on Windows and WSL. This aligns with a previous case where WSL showed notably better performance.

ChatGPT Answer TorchServe is not running on the same machine as the images; they're transferred over the network. To measure transfer time versus preprocessing time, you should log timestamps at the start and end of each stage: when the image transfer begins and ends, and when preprocessing starts and finishes. This will help pinpoint where delays are occurring.

5.5. Answering RQ$_5$ — How accurate, precise, useful, and concise are the answers from BugMentor?

The metric-based evaluation above demonstrates the benefits of our technique, BugMentor, in answering follow-up questions from bug reports. We conduct a preliminary developer study involving 23 participants to further demonstrate the benefits of BugMentor. Given a bug report (e.g., title, description) and its follow-up question, we evaluate how *accurate*, *precise*, *useful*, and *concise* (details in Table 13) our answers are according to human developers. Although the developer study provided useful practitioner feedback, we acknowledge that its scope was limited. The study was conducted using only 12 bug reports, which represent a small sample compared to the diversity and complexity of real-world software bugs. Due to this limitation, our results should not be interpreted as broadly generalizable.

Table 14
Comparison of BugMentor with the baseline techniques using a developer study.

Quality	Model	Mean	Median	Mode
Accurate	Baseline_{Lucene}	2.63	3	3
	$\text{Baseline}_{Mistral}$	2.33	2	1
	ChatGPT	2.38	2	1
	BugMentor	**3.01**	**3**	**3**
Precise	Baseline_{Lucene}	2.60	3	3
	$\text{Baseline}_{Mistral}$	2.36	2	2
	ChatGPT	2.31	2	1
	BugMentor	**3.03**	**3**	**3**
Useful	Baseline_{Lucene}	2.53	3	3
	$\text{Baseline}_{Mistral}$	2.43	2	2
	ChatGPT	2.42	2	2
	BugMentor	**3.71**	**4**	**5**
Concise	Baseline_{Lucene}	2.45	2	2
	$\text{Baseline}_{Mistral}$	2.35	2	2
	ChatGPT	2.34	2	3
	BugMentor	**3.64**	**4**	**4**

Study Participants:

For our study, we target English-speaking software developers with professional experience in four programming languages — Python, Java, JS, and C++ — that were used by our collected projects. We sent an open invitation through our personal connections, and 10 graduate students and 13 working professionals responded. Each of them had professional experience in software development and at least two years of experience in the aforementioned programming languages. We provide them with a quick overview of our project using relevant examples of bug reports, follow-up questions and answers from GitHub. We provide them a secured link https://surveys.dal.ca/opinio/s?s=75051 containing our survey on Opinio, a tool for conducting anonymous surveys. None of the participants knew the specifics of our designed technique — BugMentor.

Study Setup: For our study, we use 12 use cases where each use case consists of a bug report and a follow-up question. To select these use cases, we apply random sampling without replacement to the held-out dataset (Section 4.1). To avoid information overload, we select such bug reports that (a) do not have any stack trace information and (b) do not warrant any project-specific knowledge to understand the bug. We select three randomly sampled bug reports from each subset (based on programming language, Section 4.1) and collect corresponding follow-up questions, ground truth answers, and the answers from both BugMentor and baseline techniques — Baseline_{Lucene}, $\text{Baseline}_{Mistral}$ and ChatGPT.

We present all 12 use cases to each of the participants. Then the participants were instructed to assess the accuracy, precision, usefulness and conciseness of the generated answers (by BugMentor and baselines) with respect to the ground truth answers. We instruct the participants to submit their evaluation on a five-point Likert scale, where 1 indicates strongly disagree and 5 indicates strongly agree. **We also anonymize the source of all generated answers to avoid any potential bias towards any of the techniques.** We thus collect a total of 192 data points (12 questions ×4 explanations ×4 evaluation aspects) from each of the 23 participants.

The design of our developer survey has been reviewed and approved by the Dalhousie University Research Ethics Board (REB file #: 2023-6885).

Study Results and Discussion: Table 14 summarizes our findings from the developer study. We note that, on average, the participants found the answers from BugMentor to be the most accurate, precise, useful, and concise. Based on the median and mode values, we see that the participants agree with our answers the most. Similar to the findings in RQ_2, the participants found the closest competitor of BugMentor to be Baseline_{Lucene} across all four metrics followed by ChatGPT. According to the median values, the developers agree with Baseline_{Lucene} and BugMentor equally in terms of Accuracy and Precision.

Table 15
Manual analysis — 228 samples.

Dataset	AC	AP	AP+AddInfo	AddInfo	NR
Java	11	18	12	10	6
Python	13	21	9	9	5
C++	9	18	13	13	4
JavaScript	8	20	11	11	7
Average%	20.92%	39.29%	22.96%	21.94%	11.22%

AC = Answers Completely, **AP** = Answers Partially,
AddInfo = Additional Information,
NR = Not Relevant

Fig. 7 shows the distribution of participants' agreement levels with different quality aspects of the answers. We observe that participants find BugMentor's answers to be most useful (~60%) and most concise (~60%) compared to Baseline_{Lucene}, $\text{Baseline}_{Mistral}$ and ChatGPT. We also see that the participants strongly agree with BugMentor for a substantial part of the time (e.g., ~30% for accuracy), and strongly disagree only a few times (e.g., <20% times), which none of the baselines achieved. On the other hand, nearly half of the time, the participants disagree with AnswerBot, Baseline_{Lucene} and $\text{Baseline}_{Mistral}$ regarding various quality aspects such as Accuracy, Precision, Usefulness and Conciseness of their provided answers.

We also perform Mann–Whitney Wilcoxon test (Cuzick, 1985) to check if the developers' preferences to Baseline_{Lucene}, $\text{Baseline}_{Mistral}$ and ChatGPT are significantly lower than that of BugMentor using Bonferroni Correction (Weisstein, 2004). We find that the preference levels for BugMentor are significantly higher than all three baselines, i.e., $p = 0.0037 < 0.016$ for Baseline_{Lucene}, $p = 0.0016 < 0.016$ for $\text{Baseline}_{Mistral}$, and $p = 0.00016 < 0.016$ for ChatGPT. To quantify the magnitude of the differences, we calculate Cliff's Delta Effect sizes for BugMentor compared to BaselineLucene, Mistral, and ChatGPT are 0.79, 1 and 0.97, indicating a large effect.

Manual Analysis:

To further investigate the usefulness of BugMentor's answers, we perform a manual analysis on 228 bug reports (i.e. 57 samples for each programming language) and 976 bug reports (i.e. 244 samples for each programming language). We select these samples from the whole dataset using stratified random sampling with a 95% confidence level and 5% error margin for 228 bug reports and 97.5% confidence level and a 2.5% margin of error for 976 samples. We collect the bug reports, follow-up questions, ground truth and generated answers. For each bug report, we evaluate BugMentor's answers against the ground truth answers provided by the developer community, categorizing them into four levels of effectiveness — 'Answers Completely' indicates responses that fully address the question with accurate technical details, 'Answers Partially' represents responses that contain correct but incomplete information; 'Additional Information' denotes cases where BugMentor provided relevant supplementary details beyond the ground truth, and 'Not Relevant' identifies responses that failed to address the core question or contained incorrect information. This categorization helped us systematically assess BugMentor's performance across different types of bug reports and programming languages. The primary manual analysis for the paper was conducted by the first author and documented using an Excel sheet, with a total of ~20 h spent on the analysis. Tables 15 and 16 show the summary of our analysis.

We analyze our generated answers to the follow-up questions, contrast them against the ground truth, and determine whether they respond to the question completely, partially, or simply provide additional information or are not relevant. From Table 15, we find that BugMentor, on average, was able to answer the questions completely for 20.92% of the cases from each programming language. It was able to answer 39.29% of the questions partially, while 22.96% of

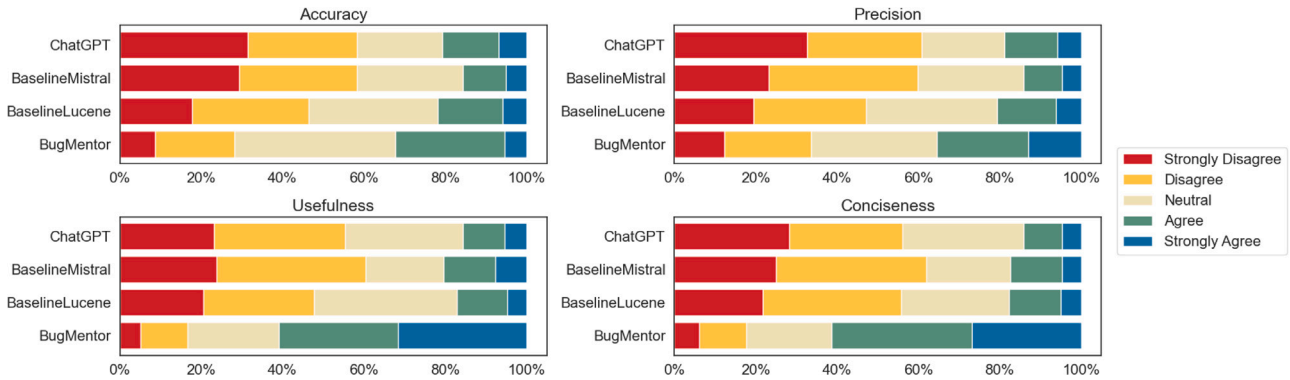

Fig. 7. Comparison of BugMentor with the baseline techniques and ChatGPT using the Likert scores.

the cases it was able to answer the questions partially while adding complementary information. Furthermore, in 21.94% cases, our technique delivered such answers that did not match with the ground truth answers but were complementary or somewhat relevant to the questions. 11.22% of the cases, BugMentor generated answers that were not relevant. From Table 16, we find that BugMentor, on average, was able to answer questions completely for 21.52% of the cases from each programming language. It was able to answer 27.66% of the questions partially and add complementary information to 22.95% of the questions. Furthermore, according to our manual analysis, in 15.68% of cases, our technique delivered such answers that did not match the ground truth answers but were complementary or somewhat relevant to the questions. For 12.19% of the cases, BugMentor's provided answers were not relevant.

While BugMentor demonstrates good performance in generating meaningful answers to follow-up questions, there are cases where it underperforms. Table 17 provides two such cases. Here, we analyze two negative cases, identify the potential causes, and suggest strategies for improvement.

In bug #2830, a Hive Task Lock issue, BugMentor struggled due to a vaguely worded follow-up question — "there is some exception info or how to reproduce this situation". The bug report also failed to include essential information such as exception logs, environment details, and reproduction steps. To handle such cases, existing work (e.g.,Imran et al. (2021)) can be leveraged to capture complementary information such as exception logs, steps to reproduce, or relevant code snippets. For cases where direct user input may not be immediately available, incorporating external knowledge sources like Stack Overflow or technical documentation can supplement the bug reports and thus can help BugMentor generate more meaningful answers.

In Bug #2071, a Kubernetes ClassCastException, BugMentor struggled to provide actionable insights due to the absence of critical information such as stack trace details in the bug report. To address such limitations, existing work (e.g., Chaparro et al. (2017)) can be leveraged to improve the handling of incomplete or ambiguous bug reports. For instance, techniques for automatically extracting or inferring missing information, such as stack traces or environment details, could be applied. Furthermore, integrating external knowledge sources like documentation could help supplement the bug report with relevant context or solutions, even when user-provided details are insufficient.

Summary of RQ_5: Developers with professional development experience found BugMentor's answers to be accurate, precise, concise, and useful in the 12 evaluated cases. Their preference levels for BugMentor were higher than those of the three baseline techniques by a statistically significant margin. However, we acknowledge that these results are based on a small set of bug reports and might not be generalizable to complex real-world bugs.

Table 16
Manual analysis — 976 samples.

Dataset	AC	AP	AP+AddInfo	AddInfo	NR
Java	61	91	34	32	26
Python	63	52	50	41	38
JavaScript	59	48	66	45	26
C++	27	79	74	35	29
Average %	21.52%	27.66%	22.95%	15.68%	12.19%

AC = Answers Completely, **AP** = Answers Partially,
AddInfo = Additional Information,
NR = Not Relevant

6. Practical implications

The practical adoption of BugMentor can be integrated into existing software development workflows in various ways, which we discuss below.

Addressing the Bottleneck in Bug Resolution BugMentor addresses a critical bottleneck in software maintenance: the timely resolution of follow-up questions in bug reports. Missing information in bug reports is a pervasive issue, with 77% of developers from major tech companies identifying it as a major problem (Zou et al., 2018). By automating the generation of relevant answers to follow-up questions, BugMentor reduces the time developers spend waiting for responses and improves the efficiency of bug resolution. This directly tackles the challenge of non-reproducibility and incomplete bug reports, which are key factors behind delayed bug fixes (Rahman et al., 2020).

Enhancing Developer Productivity Given that developers spend approximately 50% of their programming time on bug-related tasks (Britton et al., 2013), BugMentor significantly enhances productivity. It provides immediate answers to follow-up questions, reduces back-and-forth communications, and prevents premature bug closures due to missing information. This not only streamlines the bug resolution process but also allows developers to focus on higher-value tasks, ultimately improving software quality and reducing maintenance costs.

Seamless Integration into Existing Workflows BugMentor can be seamlessly integrated into existing software development workflows through multiple channels. It can be implemented as a plugin or extension for popular issue-tracking systems like GitHub Issues and JIRA. In these platforms, it automatically generates and suggests relevant answers when developers post follow-up questions on bug reports. This integration ensures that BugMentor becomes a natural part of the development process without requiring significant changes to existing workflows.

Deployment as a Microservice For organizations with custom development infrastructures, BugMentor can be deployed as a microservice accessible through REST APIs. This makes it compatible with a wide range of development tools and environments, ensuring flexibility and scalability. The system's reliance on structured information retrieval and neural text generation (e.g., Mistral) ensures that

Table 17
Underperforming examples.

Bug ID	2071	2830
Bug Report	Title: ClassCastException: class io.fabric8.kubernetes.api.model. Description: Secret cannot be cast to class io.fabric8.kubernetes.api.model.GenericKubernetesResource I have an operator where I set up watcher using the customResource to be able to watch any kind of resource, something like: javaCustomResourceDefinition Context ctx = createCRDContext(“”v1“”, “”Secret“”);	Title:Use eScheduler to Schedule the Hive Task Description: Stopping the workflow will lock the table. You need to manually delete the lock record in Hive metadata. Is there any way to avoid this problem? Thank you.
Relevant Candidate Answer	I wouldn't agree 100% on this being a bug. >If you are dealing with a known type, why do you want to handle this as a Raw/Unstructured/CustomResource type	@caishunfeng but i think re-run should keep only one workflow instance, because semantically speaking, re-run is indeed the same task. I think we just need to determine the assignment logic of the two field values of startTime and Duration...
Question	Could you please add eprintstacktrace, and check what's the exception this is showing me, this openjdk bit server vm warning	Are there any exception info or how to reproduce this situation ?
Answer	1. The bug report is not relevant to the question. 2. The question is not clear. 3. The question is not concise. 4. The question is not relevant to the bug report. 5. The question is not relevant to the relevant information. 6. The question is not relevant to the relevant information. 7. The question is not relevant to the relevant information.	The exception is related to the use of the Hive task in the workflow. When the Hive task is stopped, it will lock the table in the Hive database. This causes the workflow to fail because the table is locked. You need to manually delete the record of the lock table in the Hive metadata. There is no way to avoid this problem, but you can use...

it can handle diverse programming languages and project sizes, as demonstrated by its evaluation from top GitHub projects.

Cost-effective and Scalable Adoption BugMentor's design ensures cost-effective adoption across projects of any size. It requires minimal additional infrastructure beyond existing bug-tracking systems, making it accessible to both small teams and large organizations. The provided replication package, which includes the working prototype, experimental dataset, and configuration details, further simplifies deployment and customization. This fosters broader adoption and enables organizations to tailor BugMentor to their specific needs.

Reducing the Economic Impact of Software Bugs By automating the answering of follow-up questions, BugMentor contributes to cost-effective software maintenance. This aligns with the broader goal of reducing the economic impact of software bugs, which cost the global economy billions of dollars annually (Zou et al., 2018; Britton et al., 2013). The tool's ability to generate accurate, precise, and concise answers, as validated by developer studies, ensures that it delivers tangible value in real-world scenarios.

Facilitating Community and Industry Adoption The comprehensive evaluation of BugMentor, including its performance metrics (e.g., BLEU score, METEOR, ROUGE, Semantic Similarity) and developer studies, demonstrates its effectiveness across diverse programming languages and project sizes. This evidence-based validation, combined with the availability of a replication package, encourages adoption not only within individual organizations but also across the broader software development community. By addressing a universal pain point in software maintenance, BugMentor has the potential to become a widely adopted tool in both industry and open-source projects.

7. Related work

Question Answering (QA) has been an active research topic in both Information Retrieval (IR) and Natural Language Processing (NLP) communities (Ravichandran and Hovy, 2002; Brill et al., 2002; Waltz, 1978; Iyyer et al., 2014; Asaduzzaman et al., 2013; Tian et al., 2017; Lu et al., 2021; Bansal et al., 2021; Xu et al., 2017; Abdellatif et al., 2020). There have also been several works that focus on question-answering in the context of software engineering. Breu et al. (2010) first analyzed follow-up questions from bug reports and found that 32.34% of them were never responded to, leading to delays in fixing the bug. Their study highlighted the need for techniques to address the problem of follow-up questions and improve the bug resolution workflow. Recently, Imran et al. (2021) proposed Bug-AutoQ that recommends follow-up questions against a deficient bug report leveraging development history using information retrieval. Their technique identifies relevant follow-up questions based on the similarity between the current bug report and the most similar historical bug report. Their goal was to assist developers in gathering relevant information. However, their technique does not answer the follow-up questions.

Murgia et al. (2016) conducted a preliminary study that leveraged the search feature of the StackOverflow Q&A site to suggest relevant questions in response to error messages from version control systems like GitHub. While their technique showed promise, it was limited to providing only simple, recurring questions related to Git error messages, overlooking other types of errors or bugs. Tian et al. (2017) proposed APIBot, a system capable of answering questions related to specific APIs by analyzing the relevant API documentation. APIBot demonstrated the potential of leveraging existing documentation to provide targeted answers. However, the scope of their solution was limited to API-related questions, overlooking other problems that developers encounter during the software development process. Bansal et al. (2021) design a context-aware QA system to answer basic questions about subroutines. They designed an encoder–decoder model that extracts information directly from the source code. Lu et al. (2021) introduced a novel question-answering approach called BEAT (Bug rEport AssistanT) that aims to provide accurate and relevant answers to questions related to bug reports. Their approach leverages the structure and content of bug report templates to generate structured queries from complex questions that can be executed to retrieve relevant information effectively. However, their approach might fail when a bug report does not contain the requested information. Xu et al. (2017) designed AnswerBot to synthesize answers for technical, non-factoid questions related to Java from StackOverflow using information retrieval. However, they only use the title of a question overlooking the detailed problem context (e.g., question body), and thus their answers might be unaware of the problem context. We compare BugMentor with AnswerBot using experiments, and the detailed comparison can be found in Section 5.2. Abdellatif et al. (2020) introduced MSRBot, a system designed to answer the most common questions related to software development and maintenance. While MSRBot demonstrated the potential of mining software repositories to provide relevant answers, its effectiveness might be limited by the available information in the mined repositories. Song et al. (2022) developed BURT, a system specifically designed to support bug reporters of Android applications. While

BURT showed promise in assisting Android developers, its approach might not generalize well to other software applications, limiting its broader applicability.

Recently, Language Model-based approaches (LLMs) such as ChatGPT have emerged as a powerful text generation tool. After conducting a limited qualitative analysis (Section 5.2), we note that while ChatGPT exhibit an understanding of a given bug report, they often struggle to come up with precise answers to follow-up questions. BugMentor has a better understanding of past bug reports and thus captures a broader context. We also do a thorough qualitative and quantitative analysis to compare the performances of BugMentor and ChatGPT. We find that while ChatGPT understands the bug context, it lacks specificity and additional information that BugMentor provides. The detailed comparison can be found in Section 5.4

In summary, existing research in the field of automated question answering for software development has primarily focused on two main areas: improving deficient bug reports and addressing specific types of questions. Studies have proposed techniques to enhance the quality of bug reports by suggesting relevant questions or providing answers to inquiries related to APIs, subroutines, and Git error messages. While these contributions have been valuable, they have not directly tackled the challenge of automatically answering follow-up questions that arise from bug reports. To the best of our knowledge, our proposed technique is the first to automatically answer the follow-up questions from bug reports, which makes our work *novel*. We also combine structured information retrieval with neural text generation (e.g., Mistral) to generate the answers, which were found to be meaningful, accurate, precise, useful, and concise according to two types of evaluation — automated metrics and developer study. Our technique also outperforms four baselines from the existing literature, which demonstrates its strong potential.

We also distinguish our work from duplicate bug report detection, which focuses on identifying similar bug reports to avoid redundant maintenance efforts (Nguyen et al., 2012; Chaparro et al., 2019). Our work differs from duplicate bug detection in several aspects. First, although duplicates are identified in our work, we leverage them to generate answers against follow-up questions from deficient bug reports, which is a novel research problem. According to existing work (Rahman et al., 2020), unanswered questions pose significant maintenance challenges and lead to delayed bug resolution. Second, duplicate detection methods often rely on textual similarity or clustering techniques to identify the reports describing the same issue. In contrast, our work leverages contextual information and historical data from past, similar bug reports and employs neural text generation to synthesize meaningful responses to follow-up questions. Developers also found our answers to be useful, as shown by the developer study in Section 5.5. Finally, while duplicate bug detection aims to reduce redundancy, our work enhances the quality and completeness of bug reports by offering actionable answers to follow-up questions. Thus, our work attempts to improve bug report management by ensuring that the developers have the necessary information to triage and debug the bugs effectively.

8. Threats to validity

We identify a few threats to the validity of our findings. In this section, we examine these threats and discuss the steps that were taken to mitigate them.

External Validity: Threats to external validity refer to the lack of generalizability in the findings (Ferguson, 2004). One threat could stem from our selection of subject systems. We select 20 software systems written in four programming languages: Python, Java, JavaScript, and C++, which might not represent all systems at GitHub. However, the underlying algorithm of BugMentor is not bound to any programming language and thus can be easily adapted to any other platforms.

The second threat could stem from the small sample size of the held-out dataset for evaluation (e.g., 550). However, to mitigate this concern, we selected them carefully through random sampling from all four subsets (95% confidence level, 4.06% error margin — Section 4.1.2). We also maintain diversity in selecting our 20 subject systems (Section 4.1.2). Another threat could stem from the manual analysis; the constrained sample size may limit the generalizability of the findings. However, to mitigate this concern, we carefully random sample them from all four subsets using a 97.5% confidence level and 2.5% error margin.

Construct Validity: Construct validity refers to the extent to which the experiment measures what it intends to measure (Smith, 2005). Inappropriate use of evaluation metrics could be a threat to construct validity. However, we chose our evaluation metrics — BLEU, METEOR, Semantic Similarity, and ROUGE — based on relevant literature (Papineni et al., 2002; Banerjee and Lavie, 2005; Haque et al., 2022; Lin, 2004). We also chose the four quality aspects of generated answers based on relevant literature (Imran et al., 2021; Joshi et al., 2015). Thus, threats to construct validity might be mitigated.

Internal Validity: Threats to internal validity relate to experimental errors and subjective biases (Christ, 2007). We use manually annotated ground truth to answer both RQ1 and RQ2, which could be a source of threat. However, to mitigate this, the annotators were given appropriate training for their annotation tasks. We also employ majority voting (Kuhrmann et al., 2017) for decision-making and calculate Cohen's κ to demonstrate the agreement levels between annotators (Kuhrmann et al., 2017). In the developer study, the assessment of answers can be influenced by subjective bias. However, we anonymize the source of all answers to avoid any bias towards any technique. For our supplementary manual analysis, while it provided insights into BugMentor's answer quality, two limitations should be noted. The evaluation relied on a single rater, which may introduce subjective bias, and sample size constraints could limit generalizability.Another source of threat could be the replication of the baseline techniques. For the replication of Mistral, we collected the pre-trained model from HuggingFace[12], and for the replication of Lucene, we used ElasticSearch https://www.elastic.co/elasticsearch/, a standard library. To replicate AnswerBot (Xu et al., 2017), we used the replication package from the original authors (Bowen Xu, 2017). To replicate DialBERT (Li et al., 2020), we used the replication package from the original author. Furthermore, we followed the documentation closely for any customizations. Thus, threats to internal validity might be mitigated.

9. Conclusions and future work

Missing information in bug reports makes bug resolution challenging. Hence, developers often ask follow-up questions to gather additional information. This paper proposes BugMentor, an approach that answers the follow-up questions from bug reports combining structured information retrieval and neural text generation. Our technique leverages the relevance between past and current bug reports to gather additional context, which helps us generate an appropriate answer to the question. We evaluate the answers from BugMentor using both evaluation metrics and developer study. Our technique achieves an average BLEU Score of up to 36 and a Semantic Similarity of up to 82, demonstrating its ability to generate understandable and good answers to follow-up questions, as per Google's Standard. We further conduct a developer survey involving 23 participants to evaluate the answers. The participants found the answers from BugMentor to be more accurate, precise, concise and useful compared to the baseline answers.

There are several avenues for future work. First, we plan to design a tool encapsulating BugMentor and integrate into real-world platforms

[12] https://huggingface.co/mistralai/Mistral-7B-v0.1

like GitHub or JIRA to assist the bug reporters and the developers in their work. In particular, feedback from real-life problem contexts will be useful to improve our algorithm. Second, BugMentor does not use fine-tuned version of Mistral. If the labeled dataset can be extended, it can be used to fine-tune our designed model, which could lead to better answers.

CRediT authorship contribution statement

Usmi Mukherjee: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation. **Mohammad Masudur Rahman:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Here is the URL link to the code and data of the article — http://bit.ly/3I8l86a.

References

Abdellatif, Ahmad, Badran, Khaled, Shihab, Emad, 2020. MSRBot: Using bots to answer questions from software repositories. Empir. Softw. Eng. 25, 1834–1863.

Agarwal, Abhaya, Lavie, Alon, 2008. Meteor, m-bleu and m-ter: Evaluation metrics for high-correlation with human rankings of machine translation output. In: Proceedings of the Third Workshop on Statistical Machine Translation. pp. 115–118.

Anvik, John, Hiew, Lyndon, Murphy, Gail C., 2006. Who should fix this bug? In: Proceedings of the 28th International Conference on Software Engineering. pp. 361–370.

Asaduzzaman, Muhammad, Mashiyat, Ahmed Shah, Roy, Chanchal K, Schneider, Kevin A, 2013. Answering questions about unanswered questions of stack overflow. In: 2013 10th Working Conference on Mining Software Repositories. MSR, IEEE, pp. 97–100.

Atwood, Jeff, 2017. SQL 2008 full-text search problems. URL https://stackoverflow.blog/2008/11/01/sql-2008-full-text-search-problems/.

Banerjee, Satanjeev, Lavie, Alon, 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In: Proceedings of the Acl Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/Or Summarization. pp. 65–72.

Bansal, Aakash, Eberhart, Zachary, Wu, Lingfei, McMillan, Collin, 2021. A neural question answering system for basic questions about subroutines. In: 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, pp. 60–71.

Bodden, Eric, Sch, Wilhelm, van Deursen, Arie, Zisman, Andrea, et al., 2017. Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering: ESEC/FSE 2017, Paderborn, Germany, September 4-8, 2017. Association for Computing Machinery (ACM).

Bowen Xu, 2017. Maxxbw54/AnswerBot: Replication Package of the paper "AnswerBot: An answer summary generation tool based on Stack overflow", URL https://github.com/maxxbw54/AnswerBot.

Breu, Silvia, Premraj, Rahul, Sillito, Jonathan, Zimmermann, Thomas, 2010. Information needs in bug reports: improving cooperation between developers and users. In: Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work. pp. 301–310.

Brill, Eric, Dumais, Susan, Banko, Michele, 2002. An analysis of the askmsr question-answering system. In: Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing. EMNLP 2002, pp. 257–264.

Britton, Tom, Jeng, Lisa, Carver, Graham, Cheak, Paul, Katzenellenbogen, Tomer, 2013. Reversible Debugging Software-Quantify the Time and Cost Saved Using Reversible Debuggers. University Cambridge, Cambridge, UK.

Chaparro, Oscar, Florez, Juan Manuel, Singh, Unnati, Marcus, Andrian, 2019. Reformulating queries for duplicate bug report detection. In: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, pp. 218–229.

Chaparro, Oscar, Lu, Jing, Zampetti, Fiorella, Moreno, Laura, Di Penta, Massimiliano, Marcus, Andrian, Bavota, Gabriele, Ng, Vincent, 2017. Detecting missing information in bug descriptions. In: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. pp. 396–407.

Christ, Theodore J., 2007. Experimental control and threats to internal validity of concurrent and nonconcurrent multiple baseline designs. Psychol. Sch. 44 (5), 451–459.

Cohen, Jacob, 1988. Statistical Power Analysis for the Behavioral Sciences. Routledge.

Cuzick, Jack, 1985. A wilcoxon-type test for trend. Stat. Med. 4 (1), 87–90.

Feng, Zhangyin, Guo, Daya, Tang, Duyu, Duan, Nan, Feng, Xiaocheng, Gong, Ming, Shou, Linjun, Qin, Bing, Liu, Ting, Jiang, Daxin, et al., 2020. Codebert: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155.

Ferguson, Linda, 2004. External validity, generalizability, and knowledge utilization. J. Nurs. Sch. 36 (1), 16–22.

Furnas, George W., Landauer, Thomas K., Gomez, Louis M., Dumais, Susan T., 1987. The vocabulary problem in human-system communication. Commun. ACM 30 (11), 964–971.

Haiduc, Sonia, Bavota, Gabriele, Marcus, Andrian, Oliveto, Rocco, De Lucia, Andrea, Menzies, Tim, 2013. Automatic query reformulations for text retrieval in software engineering. In: 2013 35th International Conference on Software Engineering. ICSE, IEEE, pp. 842–851.

Haque, Sakib, Eberhart, Zachary, Bansal, Aakash, McMillan, Collin, 2022. Semantic similarity metrics for evaluating source code summarization. In: Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension. pp. 36–47.

Hu, Xing, Li, Ge, Xia, Xin, Lo, David, Jin, Zhi, 2020. Deep code comment generation with hybrid lexical and syntactical information. Empir. Softw. Eng. 25, 2179–2217.

Huang, Gao, Guo, Chuan, Kusner, Matt J, Sun, Yu, Sha, Fei, Weinberger, Kilian Q, 2016. Supervised word mover's distance. Adv. Neural Inf. Process. Syst. 29.

Husain, Hamel, Wu, Ho-Hsiang, Gazit, Tiferet, Allamanis, Miltiadis, Brockschmidt, Marc, 2019. Codesearchnet challenge: Evaluating the state of semantic code search. arXiv preprint arXiv:1909.09436.

IEEE, 1990. IEEE standard glossary of software engineering terminology. pp. 1–84. http://dx.doi.org/10.1109/IEEESTD.1990.101064, IEEE Std 610.12-1990.

Imran, Mia Mohammad, Ciborowska, Agnieszka, Damevski, Kostadin, 2021. Automatically selecting follow-up questions for deficient bug reports. In: 2021 IEEE/ACM 18th International Conference on Mining Software Repositories. MSR, IEEE, pp. 167–178.

Iyyer, Mohit, Boyd-Graber, Jordan, Claudino, Leonardo, Socher, Richard, Daumé III, Hal, 2014. A neural network for factoid question answering over paragraphs. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing. EMNLP, pp. 633–644.

Jensen, Simon Holm, Møller, Anders, Thiemann, Peter, 2009. Type analysis for JavaScript. In: Static Analysis: 16th International Symposium, SAS 2009, Los Angeles, CA, USA, August 9-11, 2009. Proceedings 16. Springer, pp. 238–255.

Jiang, Albert Q, Sablayrolles, Alexandre, Mensch, Arthur, Bamford, Chris, Chaplot, Devendra Singh, Casas, Diego de las, Bressand, Florian, Lengyel, Gianna, Lample, Guillaume, Saulnier, Lucile, et al., 2023. Mistral 7B. arXiv preprint arXiv:2310.06825.

Johnson, Jack, Mahmud, Junayed, Wendland, Tyler, Moran, Kevin, Rubin, Julia, Fazzini, Mattia, 2022. An empirical investigation into the reproduction of bug reports for android apps. In: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, pp. 321–322.

Joshi, Ankur, Kale, Saket, Chandel, Satish, Pal, D. Kumar, 2015. Likert scale: Explored and explained. Br. J. Appl. Sci. Technol. 7 (4), 396–403.

Kamphuis, Chris, de Vries, Arjen P, Boytsov, Leonid, Lin, Jimmy, 2020. Which BM25 do you mean? A large-scale reproducibility study of scoring variants. In: European Conference on Information Retrieval. Springer, pp. 28–34.

Kuhrmann, Marco, Fernández, Daniel Méndez, Daneva, Maya, 2017. On the pragmatic design of literature studies in software engineering: an experience-based guideline. Empir. Softw. Eng. 22, 2852–2891.

Kusner, Matt, Sun, Yu, Kolkin, Nicholas, Weinberger, Kilian, 2015. From word embeddings to document distances. In: International Conference on Machine Learning. PMLR, pp. 957–966.

Labrak, Yanis, Bazoge, Adrien, Morin, Emmanuel, Gourraud, Pierre-Antoine, Rouvier, Mickael, Dufour, Richard, 2024. Biomistral: A collection of open-source pretrained large language models for medical domains. arXiv preprint arXiv:2402.10373.

Lazaridou, Angeliki, Gribovskaya, Elena, Stokowiec, Wojciech, Grigorev, Nikolai, 2022. Internet-augmented language models through few-shot prompting for open-domain question answering. arXiv preprint arXiv:2203.05115.

Li, Tianda, Gu, Jia-Chen, Zhu, Xiaodan, Liu, Quan, Ling, Zhen-Hua, Su, Zhiming, Wei, Si, 2020. Dialbert: A hierarchical pre-trained model for conversation disentanglement. arXiv preprint arXiv:2004.03760.

Li, Zhenmin, Tan, Lin, Wang, Xuanhui, Lu, Shan, Zhou, Yuanyuan, Zhai, Chengxiang, 2006. Have things changed now? An empirical study of bug characteristics in modern open source software. In: Proceedings of the 1st Workshop on Architectural and System Support for Improving Software Dependability. pp. 25–33.

Lin, Chin-Yew, 2004. Rouge: A package for automatic evaluation of summaries. In: Text Summarization Branches Out. pp. 74–81.

Lu, Jinting, Sun, Xiaobing, Li, Bin, Bo, Lili, Zhang, Tao, 2021. BEAT: Considering question types for bug question answering via templates. Knowl.-Based Syst. 225, 107098.

Mahbub, Parvez, Shuvo, Ohiduzzaman, Rahman, Mohammad Masudur, 2023. Explaining software bugs leveraging code structures in neural machine translation. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE, pp. 640–652.

McCandless, Michael, Hatcher, Erik, Gospodnetić, Otis, Gospodnetić, O, 2010. Lucene in Action, vol. 2, Manning Greenwich.

Mistral AI team, 2023. Announcing Mistral 7B, Mistral AI blog post, Online. URL https://mistral.ai/news/announcing-mistral-7b,

Moreno, Laura, Bavota, Gabriele, Haiduc, Sonia, Di Penta, Massimiliano, Oliveto, Rocco, Russo, Barbara, Marcus, Andrian, 2015. Query-based configuration of text retrieval solutions for software engineering tasks. In: Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. pp. 567–578.

Mukherjee, Usmi, Rahman, Mohammad Masudur, 2025. Replication Package, URL http://bit.ly/3I8l86a,

Murgia, Alessandro, Janssens, Daan, Demeyer, Serge, Vasilescu, Bogdan, 2016. Among the machines: Human-bot interaction on social q&a websites. In: Proceedings of the 2016 CHI Conference Extended Abstracts on Human Factors in Computing Systems. pp. 1272–1279.

Ng, Jun-Ping, Abrecht, Viktoria, 2015. Better summarization evaluation with word embeddings for ROUGE. arXiv preprint arXiv:1508.06034.

Nguyen, Anh Tuan, Nguyen, Tung Thanh, Nguyen, Tien N, Lo, David, Sun, Chengnian, 2012. Duplicate bug report detection with a combination of information retrieval and topic modeling. In: Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering. pp. 70–79.

Nilsen, Kelvin D., Gao, Hong, 1995. The real-time behavior of dynamic memory management in c++. In: Proceedings Real-Time Technology and Applications Symposium. IEEE, pp. 142–153.

Papineni, Kishore, Roukos, Salim, Ward, Todd, Zhu, Wei-Jing, 2002. Bleu: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics. pp. 311–318.

Ponzanelli, Luca, Bacchelli, Alberto, Lanza, Michele, 2013. Leveraging crowd knowledge for software comprehension and development. In: 2013 17th European Conference on Software Maintenance and Reengineering. IEEE, pp. 57–66.

Rahman, Mohammad Masudur, Khomh, Foutse, Castelluccio, Marco, 2020. Why are some bugs non-reproducible?:–an empirical investigation using data fusion–. In: 2020 IEEE International Conference on Software Maintenance and Evolution. ICSME, IEEE, pp. 605–616.

Rahman, Mohammad Masudur, Roy, Chanchal K., Lo, David, 2016. Rack: Automatic api recommendation using crowdsourced knowledge. SANER, In: 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering, vol. 1, IEEE, pp. 349–359.

Ravichandran, Deepak, Hovy, Eduard, 2002. Learning surface text patterns for a question answering system. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics. pp. 41–47.

Reimers, Nils, Gurevych, Iryna, 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. arXiv preprint arXiv:1908.10084.

Saha, Ripon K, Lease, Matthew, Khurshid, Sarfraz, Perry, Dewayne E, 2013. Improving bug localization using structured information retrieval. In: 2013 28th IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 345–355.

Seonwoo, Yeon, Kim, Ji-Hoon, Ha, Jung-Woo, Oh, Alice, 2020. Context-aware answer extraction in question answering. arXiv preprint arXiv:2011.02687.

Severyn, Aliaksei, Moschitti, Alessandro, 2015. Learning to rank short text pairs with convolutional deep neural networks. In: Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 373–382.

Smith, Gregory T., 2005. On construct validity: issues of method and measurement. Psychol. Assess. 17 (4), 396.

Song, Yang, Mahmud, Junayed, Zhou, Ying, Chaparro, Oscar, Moran, Kevin, Marcus, Andrian, Poshyvanyk, Denys, 2022. Toward interactive bug reporting for (android app) end-users. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 344–356.

Stroustrup, Bjarne, 1986. An overview of C++. In: Proceedings of the 1986 SIGPLAN Workshop on Object-Oriented Programming. pp. 7–18.

Tian, Yuan, Thung, Ferdian, Sharma, Abhishek, Lo, David, 2017. APIBot: question answering bot for API documentation. In: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 153–158.

Waltz, David L., 1978. An english language question answering system for a large relational database. Commun. ACM 21 (7), 526–539.

Weisstein, Eric W., 2004. Bonferroni correction. https://mathworld.wolfram.com/.

Whissell, John S., Clarke, Charles L.A., 2011. Improving document clustering using okapi BM25 feature weighting. Inf. Retr. 14, 466–487.

White, Jules, Fu, Quchen, Hays, Sam, Sandborn, Michael, Olea, Carlos, Gilbert, Henry, Elnashar, Ashraf, Spencer-Smith, Jesse, Schmidt, Douglas C, 2023. A prompt pattern catalog to enhance prompt engineering with chatgpt. arXiv preprint arXiv:2302.11382.

Wilkins, Jessica, Rodriguez, Michael, 2024. Higher performance of mistral large on mmlu benchmark through two-stage knowledge distillation.

Xiao, Yan, Keung, Jacky, Bennin, Kwabena E., Mi, Qing, 2019. Improving bug localization with word embedding and enhanced convolutional neural networks. Inf. Softw. Technol. 105, 17–29.

Xu, Bowen, Xing, Zhenchang, Xia, Xin, Lo, David, 2017. AnswerBot: Automated generation of answer summary to developers' technical questions. In: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 706–716.

Yao, Xuchen, Van Durme, Benjamin, Callison-Burch, Chris, Clark, Peter, 2013. Answer extraction as sequence tagging with tree edit distance. In: Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. pp. 858–867.

Yu, Tingrui, Gu, Xiaodong, Shen, Beijun, 2022. Code question answering via task-adaptive sequence-to-sequence pre-training. In: 2022 29th Asia-Pacific Software Engineering Conference. APSEC, IEEE, pp. 229–238.

Yuan, Quan, Cong, Gao, Thalmann, Nadia Magnenat, 2012. Enhancing naive bayes with various smoothing methods for short text classification. In: Proceedings of the 21st International Conference on World Wide Web. pp. 645–646.

Zhang, Wen, Li, Ziqiang, Wang, Qing, Li, Juan, 2019. FineLocator: A novel approach to method-level fine-grained bug localization by query expansion. Inf. Softw. Technol. 110, 121–135.

Zhao, Yuan, He, Tieke, Chen, Zhenyu, 2019. A unified framework for bug report assignment. Int. J. Softw. Eng. Knowl. Eng. 29 (04), 607–628.

Zou, Weiqin, Lo, David, Chen, Zhenyu, Xia, Xin, Feng, Yang, Xu, Baowen, 2018. How practitioners perceive automated bug report management techniques. IEEE Trans. Softw. Eng. 46 (8), 836–862.